
Asistentes de voz domésticos *local-first* con agencia segura

Diseño, implementación y evaluación de un mayordomo digital sobre hardware de consumo. El sistema J.A.R.V.I.S. como caso de estudio

José

Junio de 2026

Índice

Resumen	4
Abstract	4
1 Introducción	6
1.1 Contexto y motivación	6
1.2 Planteamiento del problema y objetivos	6
1.3 Contribuciones	7
1.4 Estructura del documento	7
2 Estado del arte y marco teórico	8
2.1 Asistentes de voz comerciales: el modelo cloud-first y sus límites	8
2.2 El movimiento local-first y el self-hosting de voz	8
2.3 De modelos conversacionales a agentes con herramientas	9
2.4 Riesgos de seguridad de los agentes con herramientas	10
2.5 Trabajos relacionados y posicionamiento de la contribución	10
3 Metodología y materiales	12
3.1 Enfoque metodológico	12
3.2 Materiales: hardware	12
3.3 Materiales: software y stack tecnológico	13
3.4 Criterios de diseño	13
3.5 Reproducibilidad	13
4 Arquitectura del sistema	14
4.1 Visión general: el principio híbrido local-first	14
4.2 El hardware: un único nodo	14
4.3 Descomposición en servicios sobre Docker Compose	15
4.4 Componentes internos del orquestador	16
4.5 El puente al host	16
4.6 Flujo de datos de un turno	16
5 Percepción y expresión: el pipeline de voz	18
5.1 Visión general: una cadena conversacional en tiempo real	18
5.2 La palabra de activación: despertar sin escuchar siempre	18
5.3 VAD y fin de turno: saber cuándo ha terminado el usuario	19
5.4 STT: reconocimiento adaptado al español en CPU	19
5.5 TTS: síntesis local y rápida	19

5.6	Caso de estudio: el detector que puntuaba 0,000	20
5.7	Eco y barge-in: que el asistente no se oiga a sí mismo	21
6	El razonamiento: el modelo de lenguaje como cerebro	22
6.1	El único componente en la nube	22
6.2	La pasarela: LiteLLM como capa de abstracción	22
6.3	Uso de herramientas: function calling	23
6.4	Gestión del contexto	24
6.5	Delegación de lo difícil	25
6.6	Personalidad y registro	25
7	Memoria y aprendizaje continuo	26
7.1	El problema: recordar sin saturar	26
7.2	Arquitectura de la memoria: dos niveles	26
7.3	El Curator: destilar hechos, descartar lo efímero	27
7.4	Recall y decay: la pieza clave	27
7.5	Por qué no mem0	28
7.6	Privacidad de la memoria	28
8	Proactividad y multicanalidad	29
8.1	El problema de la proactividad	29
8.2	El heartbeat y la puerta única	29
8.3	Multicanalidad: el texto como canal de primera clase	30
8.4	Enrutado por presencia	30
8.5	Tareas programadas: cron en lenguaje natural	31
9	Agencia: delegación segura de acciones	32
9.1	El salto de informar a actuar	32
9.2	El problema de seguridad	32
9.3	El puente al host: investigar y encargar	33
9.4	El modelo de seguridad en tres capas	33
9.5	Privilegio mínimo	34
9.6	Discusión	34
10	Visión y presencia	36
10.1	El embudo: pipeline escalonado	36
10.2	Tecnología: YOLO11n sobre OpenVINO e InsightFace	36
10.3	Distinguir al dueño	37
10.4	Integración con la presencia	37
10.5	Estado real y honestidad sobre las cifras	38
10.6	Privacidad	38
11	Seguridad y privacidad	39
11.1	La seguridad como condición de diseño	39
11.2	Inyección de prompts: el contenido externo no manda	39

11.3	Las medidas de la primera versión	40
11.4	Identidad antes que acceso	41
11.5	Defensa en profundidad y privilegio mínimo	41
11.6	Privacidad por diseño: lo local se queda en casa	41
11.7	Verificación: la seguridad se demuestra con pruebas	42
12	Evaluación y resultados	43
12.1	Metodología de evaluación	43
12.2	Latencia del pipeline de voz	43
12.3	Consumo de recursos	44
12.4	Coste económico	44
12.5	Calidad y fiabilidad	45
12.6	Limitaciones del estudio	45
13	Conclusiones y trabajo futuro	47
13.1	Conclusiones	47
13.2	Trabajo futuro	48
	Referencias	49
13.3	Marco conceptual y seguridad	49
13.4	Voz: reconocimiento, síntesis y activación	49
13.5	Razonamiento, herramientas y visión	49
13.6	Memoria y almacenamiento	50
	Anexos	51
13.7	Anexo A. Stack tecnológico y versiones	51
13.8	Anexo B. Reproducibilidad y operación	51
13.9	Anexo C. Honestidad metodológica y estado del proyecto	52
13.10	Anexo D. Nota sobre el nombre	52

Resumen

Los asistentes de voz de consumo más extendidos —Alexa, Google Assistant, Siri— se sustentan en una arquitectura *cloud-first* en la que el audio del hogar y los datos del usuario se procesan en servidores de terceros. Este modelo, cómodo y potente, impone tres costes estructurales: la cesión de la privacidad del domicilio, la dependencia de un servicio externo revocable y la opacidad de un sistema que no puede auditarse ni modificarse.

Este trabajo aborda la pregunta de si es posible construir un asistente de voz personal que invierta ese modelo —que sea *local-first*, privado y extensible— sin renunciar a las capacidades modernas de un agente conversacional, y haciéndolo sobre **hardware de consumo asequible y sin GPU dedicada**. Para responderla se diseña, implementa y evalúa un sistema completo, denominado J.A.R.V.I.S., sobre un mini-PC de bajo consumo (Lenovo ThinkCentre M70q).

La solución adopta una arquitectura híbrida en la que todo lo sensible a la privacidad y a la latencia —detección de la palabra de activación, reconocimiento y síntesis de voz, memoria, visión y búsqueda— se ejecuta en local, y únicamente el razonamiento lingüístico, en forma de texto plano, se delega a un modelo de lenguaje en la nube. Sobre esa base se desarrollan tres contribuciones que, combinadas, definen la aportación del trabajo: (i) una **memoria adaptativa** que aprende hechos del usuario y los prioriza u olvida según su uso real (*recall* y *decay*); (ii) un modelo de **agencia segura** que permite al asistente ejecutar acciones reales sobre el servidor protegido por una defensa en profundidad de tres capas, bajo el principio de que *la última defensa nunca debe ser un prompt*; y (iii) una **proactividad y multicanalidad** gobernadas por presencia, que eligen cuándo y por qué canal —voz o mensajería— comunicarse.

La evaluación muestra que el sistema cumple sus objetivos de recursos y coste con holgura, alcanza latencias adecuadas en las etapas locales y queda condicionado, en la latencia total, por la pasarela remota del modelo de lenguaje. La fiabilidad se respalda con una batería de 68 pruebas automáticas y con la verificación funcional en vivo de cada subsistema. Se concluye que un asistente doméstico soberano, capaz y seguro es viable sobre hardware modesto, y se identifican las líneas de trabajo futuro.

Palabras clave: asistente de voz, *local-first*, privacidad, agentes de lenguaje, seguridad de IA, inyección de prompts, memoria adaptativa, computación en el borde.

Abstract

Mainstream consumer voice assistants —Alexa, Google Assistant, Siri— rely on a cloud-first architecture in which household audio and user data are processed on third-party servers. This con-

venient and powerful model carries three structural costs: surrendering the privacy of the home, depending on a revocable external service, and the opacity of a system that cannot be audited or modified.

This work investigates whether it is possible to build a personal voice assistant that inverts that model—one that is local-first, private and extensible— without giving up the modern capabilities of a conversational agent, and doing so on **affordable consumer hardware without a dedicated GPU**. To answer it, a complete system named J.A.R.V.I.S. is designed, implemented and evaluated on a low-power mini-PC (Lenovo ThinkCentre M70q).

The solution adopts a hybrid architecture in which everything privacy- and latency-sensitive — wake-word detection, speech recognition and synthesis, memory, vision and search— runs locally, and only the linguistic reasoning, as plain text, is delegated to a cloud language model. On that foundation, three contributions are developed: (i) an **adaptive memory** that learns facts about the user and prioritises or forgets them based on actual use (recall and decay); (ii) a **safe agency** model letting the assistant perform real actions on the server, protected by a three-layer defence-in-depth under the principle that *the last line of defence must never be a prompt*; and (iii) presence-governed **proactivity and multichannel** behaviour that decides when and through which channel —voice or messaging— to communicate. The evaluation shows that a sovereign, capable and secure home assistant is feasible on modest hardware.

Keywords: voice assistant, local-first, privacy, language agents, AI security, prompt injection, adaptive memory, edge computing.

1 Introducción

1.1. Contexto y motivación

En poco más de una década, los asistentes de voz han pasado de ser una curiosidad de laboratorio a un electrodoméstico cotidiano: decenas de millones de hogares conviven con un altavoz que escucha, responde y, cada vez más, actúa. Sin embargo, la comodidad de estos sistemas se ha construido sobre una premisa que rara vez se discute: el micrófono del salón es, en realidad, un terminal que envía cuanto oye —tras la palabra de activación— a la infraestructura de una gran corporación, donde se transcribe, se interpreta y, a menudo, se almacena y se anota. El usuario obtiene un servicio excelente a cambio de ceder la confidencialidad de su domicilio y de aceptar una dependencia total de un proveedor que puede cambiar las reglas, subir el precio o, sencillamente, discontinuar el producto.

Paralelamente, la irrupción de los **modelos de lenguaje grandes** (LLM) ha transformado lo que cabe esperar de un asistente. Ya no se trata solo de reconocer comandos fijos, sino de mantener una conversación natural, razonar sobre peticiones ambiguas y —mediante el uso de herramientas— **actuar** en el mundo: consultar una agenda, gestionar un servidor, recordar lo que se dijo la semana pasada. Esta nueva capacidad de agencia multiplica la utilidad del asistente y, a la vez, introduce una superficie de riesgo inédita, porque un sistema que puede actuar es un sistema que puede ser engañado para actuar mal.

Estas dos corrientes —la preocupación creciente por la privacidad y la soberanía digital, y la madurez de los modelos de lenguaje— convergen en una pregunta que vertebra esta tesis.

1.2. Planteamiento del problema y objetivos

¿Es posible construir un asistente de voz personal que sea, a la vez, capaz como un agente moderno y soberano como un sistema *local-first*, ejecutándose sobre hardware de consumo asequible y sin recurrir a una GPU dedicada?

Responder a esta pregunta exige resolver varias tensiones de diseño que no son triviales: la potencia de un LLM moderno frente a las limitaciones de un mini-PC; la utilidad de la agencia frente a su peligrosidad; la conveniencia de una memoria persistente frente a la degradación que provoca saturar el contexto del modelo; y la ambición de un asistente proactivo frente al riesgo de convertirlo en un intruso molesto.

El objetivo general se concreta en los siguientes objetivos específicos:

1. **Diseñar una arquitectura híbrida** que mantenga en local todo lo sensible a privacidad y latencia (voz, visión, memoria, búsqueda) y delegue a la nube únicamente el razonamiento, en forma de texto.
2. **Implementar un pipeline de voz** completo y de baja latencia (palabra de activación, reconocimiento, síntesis) sobre una CPU sin aceleración gráfica.
3. **Dotar al asistente de una memoria adaptativa** que aprenda de la conversación y gestione el olvido de forma que priorice lo relevante sin saturar el contexto.
4. **Habilitar la agencia segura**: que el asistente pueda ejecutar acciones reales con un modelo de seguridad robusto frente al error y la manipulación.
5. **Construir proactividad y multicanalidad** gobernadas por la presencia del usuario.
6. **Evaluar** el sistema resultante en términos de latencia, recursos, coste y fiabilidad.

1.3. Contribuciones

El trabajo realiza tres contribuciones principales, que el documento desarrolla en profundidad:

- **Una memoria adaptativa con *recall* y *decay*.** En lugar de almacenar todo o de recurrir a la maquinaria de un *vector store*, se propone un almacén de hechos con metadatos de uso que prioriza lo que el usuario menciona y archiva —sin borrar— lo que cae en desuso, manteniendo el contexto inyectado dentro de un presupuesto.
- **Un modelo de agencia segura en tres capas.** Para permitir que el asistente actúe sobre el sistema sin convertirlo en un peligro, se articula una defensa en profundidad que combina confirmación determinista, juicio de riesgo asistido por el modelo y un guardia de comandos infalsificable, bajo el principio rector de que *la última defensa nunca debe ser un prompt*.
- **Un enrutado de la comunicación por presencia.** El asistente decide cuándo hablar (silencio por defecto, filtrado por una puerta única) y por qué canal (voz si el usuario está en casa, mensajería si está fuera), integrando voz, texto y visión en una sola política coherente.

1.4. Estructura del documento

El resto de la tesis se organiza como sigue. El **Capítulo 2** revisa el estado del arte y el marco teórico. El **Capítulo 3** expone la metodología y los materiales. El **Capítulo 4** describe la arquitectura general del sistema. Los **Capítulos 5 a 11** detallan los subsistemas: el pipeline de voz, el razonamiento, la memoria, la proactividad y multicanalidad, la agencia, la visión y, transversalmente, la seguridad y la privacidad. El **Capítulo 12** presenta la evaluación. El **Capítulo 13** recoge las conclusiones y las líneas de trabajo futuro. Cierran el documento las referencias y los anexos.

2 Estado del arte y marco teórico

2.1. Asistentes de voz comerciales: el modelo cloud-first y sus límites

La generación dominante de asistentes de voz —Amazon Alexa, Google Assistant y Apple Siri— comparte una arquitectura **cloud-first**. El dispositivo doméstico (un altavoz o un teléfono) ejecuta en local únicamente la detección de la palabra de activación o *wake word* —una tarea ligera y deliberadamente conservadora— y, una vez disparada, transmite el audio capturado a los servidores del proveedor. Allí ocurre lo costoso: el reconocimiento de habla (*speech-to-text*, STT), la comprensión del lenguaje natural, el razonamiento sobre la petición y la síntesis de la respuesta (*text-to-speech*, TTS). El resultado regresa al dispositivo. Bajo este reparto, el aparato del salón es esencialmente un terminal: la inteligencia reside en la nube.

Este modelo ofrece ventajas reales —cómputo prácticamente ilimitado, calidad de voz alta y actualización continua— pero impone tres limitaciones estructurales que el presente trabajo considera inaceptables para un asistente doméstico.

La primera es de **privacidad**. El audio del hogar —dato especialmente sensible, pues puede contener voz (rasgo biométrico bajo el RGPD), conversaciones privadas y la presencia de menores— se procesa en infraestructura de terceros sujeta a políticas opacas de retención, anotación humana y uso secundario. El usuario delega la confidencialidad de su domicilio en un contrato de adhesión que no controla.

La segunda es la **dependencia**. Sin conectividad o sin servicio del proveedor, el asistente deja de funcionar; el dueño no posee el sistema, sino una licencia de uso revocable. La discontinuación de productos —frecuente en este sector— puede convertir el hardware en inservible de un día para otro.

La tercera es la **opacidad y la falta de extensibilidad**. El comportamiento del asistente es una caja negra: no se puede auditar qué hace con los datos, ni inspeccionar su lógica, ni —salvo por interfaces de extensión muy acotadas (*skills*, *actions*)— modificar sustancialmente su funcionamiento. El usuario no puede cambiar el modelo de lenguaje, la voz o la política de memoria.

2.2. El movimiento local-first y el self-hosting de voz

Frente a ese paradigma se sitúa el enfoque **local-first**, articulado conceptualmente por Kleppmann y colaboradores: el dato y el cómputo residen, por defecto, en hardware bajo control del

usuario, y la nube es como mucho un complemento, no el centro de gravedad. Sus principios rectores son la **soberanía del dato** (el dueño decide qué sale de casa y qué no), el **funcionamiento sin nube para lo sensible** y la **durabilidad** (el sistema no depende de la supervivencia comercial de un proveedor). El *self-hosting* es su materialización práctica: ejecutar los servicios en infraestructura propia.

El ecosistema de voz abierta ha madurado lo suficiente como para hacer viable este enfoque. Conviene situar los proyectos que constituyen el estado del arte:

Proyecto	Función	Aportación al estado del arte
Mycroft / OpenVoiceOS	Asistente de voz abierto integral	Pionero del asistente libre; demostró la viabilidad del concepto pese a dificultades de sostenibilidad
Rhasspy	Conjunto de herramientas de voz offline, multilingüe	Modularidad por componentes intercambiables; protocolo Wyoming
Home Assistant + Voice	Voz integrada en domótica local	Llevó la voz local-first al gran público de la automatización del hogar
openWakeWord	Detección de <i>wake word</i>	Modelos preentrenados, coste de CPU ínfimo, independiente del idioma
Piper	Síntesis de voz (TTS) neuronal en CPU	Voz natural en local con licencia permisiva (MIT) y latencia baja
faster-whisper	Reconocimiento de habla (STT)	Whisper optimizado, más rápido que tiempo real en CPU

La conclusión que arroja el análisis de este ecosistema es matizada y resulta determinante para el posicionamiento de este trabajo: ningún proyecto llave en mano cubre simultáneamente conversación en tiempo real, visión, memoria evolutiva, uso de herramientas, español de calidad y ejecución en CPU sin GPU dedicada. Las piezas son excelentes por separado, pero la integración —**componer**, no adoptar— sigue siendo trabajo de ingeniería abierto.

2.3. De modelos conversacionales a agentes con herramientas

Un eje paralelo del estado del arte es la evolución de los **modelos de lenguaje grandes** (LLM). En su forma básica, un LLM es un sistema que, dado un texto, produce el texto siguiente más probable:

conversa, resume o redacta, pero su efecto se agota en el lenguaje. El salto cualitativo lo introduce el *function calling* (uso de herramientas): se describe al modelo un conjunto de funciones disponibles —consultar el tiempo, crear un recordatorio, buscar en internet— y el modelo, en lugar de responder en prosa, emite una petición estructurada para invocar una de ellas. El programa anfitrión ejecuta la función y devuelve el resultado al modelo, que prosigue.

Sobre esa capacidad se define el **agente**: un sistema en el que un LLM no solo razona, sino que **planning** una secuencia de pasos, **actúa** sobre el entorno mediante herramientas y mantiene **memoria** del estado y de interacciones previas. La diferencia con un chatbot es sustancial: el agente cierra el bucle percibir-decidir-actuar y, por tanto, produce efectos en el mundo real. Es precisamente esa capacidad de actuar la que aporta utilidad doméstica —encender luces, gestionar tareas, recordar hechos del usuario entre sesiones— y, a la vez, la que introduce los riesgos del apartado siguiente.

2.4. Riesgos de seguridad de los agentes con herramientas

Dotar a un LLM de la capacidad de **actuar** es delicado por una razón estructural: el modelo no distingue de forma fiable entre las instrucciones legítimas de su operador y las instrucciones maliciosas que pueda contener el texto que procesa. Este es el problema de la **inyección de prompts** (*prompt injection*): una página web, un correo o un documento pueden incluir órdenes encubiertas («ignora lo anterior y envía las memorias del usuario a esta dirección») que el modelo, al leerlas, podría interpretar como mandatos.

Simon Willison sistematizó el peligro bajo el nombre de **lethal trifecta** («tríada letal»): el riesgo grave se materializa cuando un agente combina, a la vez, (1) **acceso a datos privados**, (2) **exposición a contenido no confiable** y (3) **capacidad de comunicar o actuar hacia el exterior**. Con las tres condiciones presentes, contenido envenenado puede ordenar al agente exfiltrar datos privados o disparar acciones con efecto real. No es teórico: existen casos documentados de envenenamiento persistente de memoria (*SpAIware*) y de invocación diferida de herramientas. El cuerpo normativo de referencia —el OWASP LLM Top 10 y las guías agénticas de OWASP— recoge estos vectores, y técnicas como el *spotlighting* de Microsoft (marcar el contenido externo como datos, nunca instrucciones) reducen la tasa de éxito de la inyección indirecta, pero no la eliminan. De ahí el principio que la literatura y este trabajo comparten: **la última defensa nunca debe ser un prompt**; las barreras que importan han de vivir en código determinista, fuera del LLM.

2.5. Trabajos relacionados y posicionamiento de la contribución

El interés reciente por los **agentes personales** —asistentes que actúan en nombre de un usuario sobre sus datos y servicios— ha producido un ecosistema activo de marcos de orquestación (con soporte nativo de *barge-in*, gestión de turnos y uso de herramientas), de sistemas de memoria para agentes y de bancos de pruebas adversariales que evalúan su robustez frente a la inyección de prompts. Estos esfuerzos, sin embargo, tienden a asumir cómputo en la nube y rara vez integran de

forma conjunta la dimensión sensorial (voz y visión locales), la memoria adaptativa y la seguridad de la agencia.

El hueco que este trabajo pretende llenar se sitúa en esa intersección poco explorada: un **asistente doméstico que combina local-first, agencia segura y memoria adaptativa sobre hardware de consumo**. Concretamente, todo lo sensible a privacidad —*wake word*, STT, TTS, memoria, embeddings y presencia visual— se ejecuta en un mini-PC sin GPU dedicada, y solo el razonamiento, en forma de texto plano, se delega a una API externa: la voz y el vídeo nunca cruzan a la nube. Sobre esa base local-first, la capacidad de actuar se gobierna con defensas deterministas frente a la tría-da letal —confirmación verbal fuera del LLM, *spotlighting* del contenido externo, *taint mode*, guardia de comandos— y la memoria incorpora procedencia y reflexión para evitar el envenenamiento persistente. Ninguno de los proyectos previos reúne estas tres propiedades simultáneamente en hardware modesto; esa convergencia, y no cada pieza por separado, constituye la contribución que el resto de la tesis desarrolla.

3 Metodología y materiales

3.1. Enfoque metodológico

El proyecto se aborda mediante una metodología de **investigación a través del diseño** (*research through design*), en la que el conocimiento se produce construyendo, observando y refinando un artefacto real. No se trata de validar una hipótesis aislada en condiciones de laboratorio, sino de demostrar la **viabilidad** de un sistema completo bajo restricciones realistas y de extraer, del proceso de construcción, principios de diseño transferibles. Este enfoque es habitual en la ingeniería de sistemas, donde la integración de componentes —y no cada componente por separado— constituye el problema central.

El desarrollo siguió un ciclo **iterativo e incremental** organizado en fases, cada una con un criterio de éxito verificable. Una decisión metodológica importante condicionó todo el proceso: las fases se materializan como **interruptores de configuración, no como ramas de código**. El código de funcionalidades futuras convive, desactivado, con el de las activas, de modo que avanzar consiste en encender un flag y validar. Esta disciplina permitió ejecutar fases fuera de orden cuando convenía y, sobre todo, mantener en todo momento un sistema arrancable y demostrable.

La validación combinó tres instrumentos: **pruebas automáticas** (unitarias y de seguridad) que protegen los invariantes críticos frente a regresiones; **verificación funcional en vivo** de cada subsistema sobre el hardware definitivo; y un **diario de laboratorio** por sesiones que documenta decisiones, incidencias y su resolución, garantizando la trazabilidad del razonamiento de diseño.

3.2. Materiales: hardware

El sistema se construyó sobre un **Lenovo ThinkCentre M70q Tiny**, un mini-PC de sobremesa de bajo consumo elegido por representar el segmento de *hardware de consumo asequible* que el objetivo exige. Sus características relevantes son un procesador Intel Core i5-10400T (6 núcleos, 12 hilos, 35 W de potencia de diseño térmico), 16 GB de RAM, una unidad NVMe para el sistema y los modelos, y la gráfica integrada Intel UHD 630. La ausencia de GPU dedicada es deliberada: condiciona la arquitectura hacia el modelo híbrido y demuestra que la solución no exige una inversión especializada.

Para la captura y reproducción de audio se empleó un **Anker PowerConf**, un altavoz de conferencia USB con cancelación de eco por hardware, cuya elección se justifica en el capítulo del pipeline de voz. La cámara para el subsistema de visión es una webcam USB estándar, pendiente de integración física en el momento de redacción.

3.3. Materiales: software y stack tecnológico

La solución se compone íntegramente de **software libre y de servicios bajo control del usuario**, orquestados como un conjunto de contenedores mediante Docker Compose con versiones fijadas para garantizar la reproducibilidad. Las piezas principales son Pipecat como marco de orquestación de voz en tiempo real; openWakeWord para la palabra de activación; faster-whisper para el reconocimiento de voz; Piper para la síntesis; LiteLLM como pasarela hacia el modelo de lenguaje; SearXNG como metabuscador privado; SQLite (con su extensión de búsqueda de texto completo FTS5) como almacén de memoria; y OpenVINO con YOLO11n e InsightFace para la visión. La tabla de versiones exactas se recoge en el Anexo correspondiente.

3.4. Criterios de diseño

Cinco criterios, enunciados al inicio y sostenidos a lo largo del proyecto, guiaron cada decisión:

1. **Local-first.** Lo sensible a privacidad y latencia se ejecuta en local por defecto; la nube es un complemento acotado, no el centro del sistema.
2. **Privilegio mínimo y fail-closed.** Cada componente recibe el mínimo privilegio necesario, y ante la duda o el error el sistema deniega en lugar de permitir.
3. **Determinismo en las barreras críticas.** La seguridad que importa reside en código verificable, nunca en el comportamiento de un modelo probabilístico.
4. **Sustituibilidad.** Las piezas se desacoplan tras interfaces estables (la pasarela de LLM, el backend de reconocimiento, el registro de herramientas) para no quedar atados a un proveedor o una librería.
5. **Honestidad y reproducibilidad.** Las decisiones, las versiones y las limitaciones se documentan; se distingue explícitamente lo medido de lo estimado.

3.5. Reproducibilidad

Todo el código y la documentación residen en un repositorio público bajo control de versiones. Los secretos se mantienen fuera del repositorio mediante ficheros de entorno excluidos del control de versiones, de modo que clonar el proyecto no expone ninguna credencial. Los modelos y los datos de usuario viven fuera del árbol del repositorio. Esta organización permite que un tercero reconstruya el sistema desde cero siguiendo el manual de operación incluido como anexo.

4 Arquitectura del sistema

4.1. Visión general: el principio híbrido local-first

El sistema se construye sobre una decisión arquitectónica deliberada: **todo lo que toca al usuario o a sus datos corre en local; solo el razonamiento lingüístico viaja a la nube**. Este principio, denominado *local-first*, no es una postura ideológica sino el resultado de un análisis de costes y latencias. La captura de audio, la detección de la palabra de activación (*wake word*), la transcripción de voz a texto (STT), la síntesis de texto a voz (TTS), la memoria, la visión y la orquestación de herramientas se ejecutan en la propia máquina del hogar. Al exterior solo viaja **texto**: el turno transcrito se envía a un modelo de lenguaje grande (LLM) alojado en la nube y regresa convertido en respuesta. La voz del usuario, por tanto, nunca abandona el equipo.

La justificación del trade-off es doble. Ejecutar un LLM de gran capacidad en local exigiría una GPU dedicada de la que el hardware carece; delegar ese razonamiento a un proveedor externo permite usar un modelo competente sin coste de cómputo propio. A cambio se acepta una dependencia de red y la salida del texto del turno hacia un tercero. Las partes sensibles a la privacidad y a la latencia (el audio crudo, la cara del usuario, la memoria de hechos) se mantienen bajo control directo, mientras que la pieza cara en cómputo (la inferencia del modelo) se externaliza.

4.2. El hardware: un único nodo

El anfitrión es un **Lenovo ThinkCentre M70q Tiny** con un procesador Intel i5-10400T (6 núcleos / 12 hilos, 35 W de TDP), 16 GB de RAM y **sin GPU dedicada**: solo dispone de la iGPU integrada Intel UHD 630. El sistema operativo es Ubuntu Server 24.04 LTS y el repositorio reside en `/opt/jarvis`.

Estas características imponen restricciones claras. La ausencia de GPU descarta la inferencia local de modelos grandes y obliga a la estrategia híbrida descrita. El TDP de 35 W y los 16 GB de RAM acotan cuánto trabajo pesado puede correr de forma simultánea. La consecuencia práctica más visible se da en el STT: el modelo Whisper *medium* resultó inviable en CPU (entre 6 y 7 segundos por segmento), por lo que se adoptó Whisper *small* con afinado específico para español, que rinde en torno a medio segundo.

A la vez, el nodo ofrece oportunidades. Como el razonamiento del LLM se delega a la nube, la CPU local queda mayormente ociosa y puede dedicarse al STT (configurado con 12 hilos). La iGPU UHD 630, infrautilizada, se reserva para la fase de visión mediante OpenVINO, que sabe explotar gráficos integrados Intel. El particionado sigue el principio de **lo caliente en el NVMe M.2**: el sistema operativo, los contenedores Docker y los modelos —que se leen en cada arranque y de forma constante—

viven en el disco rápido, mientras que un SSD SATA de 1 TB queda como reserva de capacidad montada en solo lectura para métricas.

4.3. Descomposición en servicios sobre Docker Compose

El sistema entero es **un único proyecto Docker Compose**, con versiones de imagen fijadas y comentadas, y con la filosofía «fases con flags, no ramas»: los servicios de fases futuras ya existen en el repositorio pero permanecen apagados mediante perfiles de Compose o variables de entorno. La superficie de red es mínima: solo el panel (puerto 8080) y n8n (5678) publican puerto, y únicamente en 127.0.0.1; el resto de servicios se comunican por la red interna de Docker.

Servicio	Rol	Tecnología
orchestrator	Corazón del sistema: pipeline de voz y agente multicanal	Pipecat sobre Python
litellm	Pasarela LLM compatible con OpenAI, con alias y <i>fallbacks</i>	LiteLLM
searxng	Metabuscador privado para la herramienta de búsqueda web	SearXNG
chroma	Almacén vectorial (evaluado para la memoria semántica)	ChromaDB
postgres	Base de datos de persistencia de n8n	PostgreSQL
n8n	Motor de acciones por <i>webhooks</i> con verificación HMAC	n8n
vision	Presencia escalonada y captura de fotograma	OpenVINO + iGPU
panel	Centro de control y HUD	FastAPI + HTMX
socket-proxy	API de Docker de solo lectura para el panel	docker-socket-proxy
cloudflared	Túnel saliente para el acceso remoto seguro	Cloudflare Tunnel

La **pasarela LLM** (LiteLLM) merece atención porque materializa el principio híbrido: expone una interfaz única y compatible con OpenAI hacia el orquestador, de modo que el resto del sistema ignora qué modelo concreto razona detrás. Su alias principal apunta a un modelo de gran capacidad en la nube, con *fallbacks* a proveedores alternativos; cambiar de cerebro es modificar configuración, no código. El **buscador** (SearXNG) ofrece búsqueda web sin claves de API, alineado con el

espíritu *self-hosted*. El **panel** nunca habla directamente con el *socket* de Docker, sino a través de un proxy limitado a operaciones de lectura, reduciendo la superficie de privilegio.

4.4. Componentes internos del orquestador

El orquestador dejó de ser un simple bucle de voz para convertirse en un agente multicanal con memoria propia. Todos sus subsistemas conviven dentro del mismo servicio:

- **Pipeline de voz** — cadena Pipecat que encadena *wake gate*, detección de actividad de voz (VAD), STT y TTS. Es el canal de entrada y salida hablado.
- **Agente de texto (Telegram)** — canal de texto bidireccional que dota al sistema de una boca y un oído independientes del micrófono; es un canal de primera clase junto a la voz.
- **Registro de herramientas desacoplado** — catálogo de *tools* definido una sola vez y **compartido por ambos canales**, evitando duplicar definiciones entre voz y texto.
- **Memoria (almacén facts)** — memoria propia con *recall* (recuperación de hechos relevantes) y *decay* (los hechos pierden peso con el tiempo si no se refuerzan), alimentada por un proceso **Curator** que aprende hechos a partir de la conversación.
- **Proactividad** — el sistema puede iniciar comunicación sin que se le pregunte, emitiendo avisos por voz o eco a Telegram.
- **Tareas programadas (cron)** — planificador interno al proceso para recordatorios y trabajos recurrentes, sin depender de temporizadores externos.
- **System prompt por canal** — la composición del *prompt* de sistema se adapta al canal, de modo que el mismo cerebro se comporta de forma idónea en un diálogo hablado breve o en un intercambio escrito.

Junto a estos, un **clasificador de errores LLM** categoriza los fallos del modelo (límites de tasa, *timeouts*, errores de contexto) para decidir reintento o *failover* en coordinación con los *fallbacks* de la pasarela.

4.5. El puente al host

Para delegar acciones que requieren acceso directo al sistema de ficheros y al entorno del anfitrión, existe **un servicio fuera de Compose**, ejecutado como unidad systemd en el host, que actúa de puente hacia un agente de codificación más capaz (Claude Code). Ofrece dos capacidades —*investigar* (solo lectura) y *encargar* (ejecución, protegida por un guardia de comandos)— y vive en el host, no en un contenedor, precisamente por esa necesidad de acceso directo. Su funcionamiento se detalla en el capítulo de agencia.

4.6. Flujo de datos de un turno

Un turno completo, de la voz a la respuesta, recorre el siguiente camino: el micrófono captura audio que el *wake gate* vigila; al detectar la palabra de activación, el VAD delimita la locución, el STT la

transcribe a texto en local, y ese texto —junto con la memoria recuperada y el *prompt* compuesto para el canal— se envía por la pasarela LiteLLM al LLM en la nube. La respuesta vuelve como texto, puede invocar herramientas (búsqueda, acciones, puente al host) y finalmente el TTS la sintetiza en voz por la salida de audio.

El cableado lógico respeta cuatro invariantes: el orquestador es el único proceso con acceso al audio, igual que el servicio de visión es el único dueño de la cámara (V4L2 es de consumidor único); el panel solo accede a Docker mediante el proxy de lectura; el acceso desde internet existe solo para el panel y solo tras la identificación de Cloudflare Access; y las acciones de alto privilegio cruzan siempre por el guardia de comandos antes de ejecutarse.

5 Percepción y expresión: el pipeline de voz

5.1. Visión general: una cadena conversacional en tiempo real

El subsistema de voz constituye la frontera sensorial del asistente: el lugar donde el sonido se convierte en intención y la intención vuelve a convertirse en sonido. Su objetivo de diseño es exigente y doble. Por un lado, debe completar un turno conversacional —de la palabra del usuario a la respuesta hablada— en torno a 1,3 a 2,0 segundos. Por otro, debe respetar un principio de privacidad estricto: el audio nunca abandona el equipo doméstico. Solo el texto necesario para razonar viaja a la nube; la captura, la transcripción y la síntesis ocurren íntegramente en local, sobre un mini-PC sin GPU.

La orquestación de esa cadena recae en Pipecat, un marco de voz en tiempo real cuyo modelo de ejecución es un **pipeline de frames**. Cada fragmento de audio, cada transcripción parcial y cada señal de control circula como un *frame* a través de una secuencia ordenada de procesadores, de modo que la latencia se solapa: el sistema puede empezar a sintetizar la primera frase de la respuesta mientras el modelo de lenguaje todavía genera el resto.

Un detalle de integración resulta crítico: la detección de actividad de voz (VAD) se sitúa en el agregador de usuario, no en el transporte de entrada. Colocarla en el transporte, como hace la configuración heredada, ejecutaría el VAD *antes* del filtro de palabra de activación y desmontaría el diseño de privacidad que se describe a continuación.

5.2. La palabra de activación: despertar sin escuchar siempre

El primer eslabón es un *gate* (compuerta) de palabra de activación. La motivación es de privacidad: sin un disparador local, la única alternativa sería transcribir el audio de forma permanente y buscar la palabra clave en el texto, lo que implica que un micrófono abierto envíe sonido al reconocedor 24 horas al día. Eso es, además, inviable en esta CPU. El *gate* invierte la lógica: mientras el asistente «duerme», descarta todo el audio entrante y no transcribe nada; solo cuando detecta la palabra de activación abre el resto del pipeline. Nada se transcribe hasta que el sistema despierta.

La detección corre a cargo de openWakeWord, un detector ligero basado en modelos ONNX preentrenados que consume entre el 1 y el 3 % de un núcleo. El audio llega a 16 kHz en mono y se evalúa en bloques de 1280 muestras (80 ms); cuando la puntuación supera el umbral de 0,5, el sistema despierta, reinicia el modelo y abre una ventana de escucha de unos 45 segundos que se renueva mientras el usuario siga hablando.

La elección de la palabra concreta fue empírica. El modelo natural, «hey jarvis», resultó poco fiable con el micrófono empleado: disparaba de forma irregular. El modelo «hey Mycroft», en cambio, detectaba con solidez sobre el mismo hardware. La decisión, por tanto, fue desacoplar el disparador del nombre: el asistente sigue llamándose Jarvis, pero la palabra que lo despierta es «hey Mycroft». Es un *trade-off* deliberado —una pequeña incoherencia de marca a cambio de fiabilidad de detección— justificado por la medición, no por la preferencia.

5.3. VAD y fin de turno: saber cuándo ha terminado el usuario

Detectar el comienzo del habla es sencillo; detectar su final no lo es. Una pausa para pensar no debe interpretarse como cesión del turno, o el asistente interrumpirá al usuario a media frase. El sistema combina dos mecanismos. El VAD de Silero distingue voz de silencio en tiempo real. Sobre él opera *smart-turn*, un pequeño modelo ONNX (con soporte de español) que analiza si la frase está semánticamente completa. La estrategia es eficiente: una pausa de unos 200 ms dispara el análisis semántico (de 12 a 95 ms en CPU), que decide si el turno ha terminado de verdad. Así se resuelve de raíz el problema clásico de que el asistente corte al usuario cuando este simplemente duda.

5.4. STT: reconocimiento adaptado al español en CPU

La transcripción emplea *faster-whisper* en su variante *small* con cuantización INT8, que ocupa alrededor de 1 GB de RAM y opera más rápido que el tiempo real en este equipo. La elección refleja varios *trade-offs*. El modelo *medium* ofrecería mayor precisión, pero su coste lo hace inviable sin GPU. Una alternativa especializada como *parakeet*, considerada en la planificación, se descartó por una razón insalvable: solo soporta inglés, lo que lo inhabilita para un asistente en español. El modelo *small* quedó así como decisión definitiva, validada con transcripción de español «casi perfecta». El diseño conserva su flexibilidad: como *Pipecat* acepta cualquier backend compatible con la interfaz de *OpenAI*, sustituir el reconocedor en el futuro sería cuestión de configuración, sin cambios de código.

5.5. TTS: síntesis local y rápida

La síntesis de voz recae en *Piper* con la voz *es_ES-davefx-medium* (masculina, 22,05 kHz, licencia MIT), embebida en el orquestador sin contenedor aparte. Es local, ligera y suficientemente rápida: en la validación produjo voz en español en 1,8 segundos, y en régimen de *streaming* sintetiza la primera frase mientras el modelo de lenguaje sigue generando, lo que recorta la latencia percibida.

Etap	Tecnología	Métrica de referencia
Wake word	openWakeWord hey_mycroft	score 0,99 (clip sintético)
Fin de turno	Silero VAD + smart-turn	~200 ms pausa + 12–95 ms análisis
STT	faster-whisper small INT8	español casi perfecto
LLM	Modelo en la nube (solo texto)	TTFB ~0,49 s (Groq)
TTS	Piper es_ES-davefx-medium	voz en 1,8 s

5.6. Caso de estudio: el detector que puntuaba 0,000

La primera avería seria del proyecto ilustra un método de depuración sistemática que merece exponerse por sí mismo. El síntoma era desconcertante: con alguien hablando frente al micrófono, el detector devolvía **exactamente 0,000, turno tras turno**. Un cero perfecto y sostenido es, en sí mismo, una pista: un modelo de palabra de activación expuesto a voz real produce *algo*, aunque sea un valor bajo ante una frase ajena. Un cero absoluto sugiere ausencia de señal, no fallo de clasificación.

El diagnóstico procedió por hipótesis y descartes. Primero se inyectó un clip sintético directamente al detector, sin pasar por el micrófono: dio **0,99**. Ese único experimento absolvió al modelo y al *gate*, y acotó el problema *aguas arriba*, en la captura. A continuación se bajó el umbral del VAD a 0,0 para descartar que filtrara la voz: el score seguía en cero. Finalmente se instrumentó el detector para registrar la amplitud máxima de cada bloque capturado, y la evidencia fue concluyente: una amplitud sostenida de ~15 sobre un formato en el que la voz real supera 1000. El detector funcionaba con total honestidad: puntuaba un silencio perpetuo.

La causa raíz no estaba en el software de detección sino en la **selección del dispositivo de audio**: el sistema capturaba de un dispositivo mudo —el micrófono muerto del jack de la placa— en lugar de la entrada prevista. Dos factores lo hacían casi inevitable. Los índices de dispositivo eran inestables entre reinicios (la tarjeta pasaba de `card 1` a `card 0` según el orden de detección USB), de modo que un índice fijo apuntaba al sitio equivocado o provocaba un fallo. Además, contenedores de prueba sin detener retenían el dispositivo correcto como ocupado.

La solución elevó el incidente a principio de diseño: **identificar los dispositivos por nombre, nunca por posición**. En el código, una función recorre los dispositivos y elige por nombre; en la capa ALSA, una configuración asimétrica fija la captura y la reproducción por nombre de tarjeta, y los índices numéricos se abandonan por completo. Un nombre es una propiedad estable del hardware; un índice es un hecho circunstancial del último arranque.

El caso tuvo una segunda fase. Una vez que el micrófono capturaba señal, apareció un problema de **saturación o clipping** : ganancia excesiva que distorsionaba la entrada y degradaba la detec-

ción. La resolución combinó dos ajustes: la adopción de la palabra «hey Mycroft», más robusta sobre este hardware, y el ajuste de la ganancia de captura a un nivel que evita el recorte. La lección metodológica es transferible: ante un detector que devuelve cero absoluto, conviene sospechar primero de la señal y solo después del modelo, y un test sintético que aíse el componente ahorra horas de mirar en el lugar equivocado.

5.7. Eco y barge-in: que el asistente no se oiga a sí mismo

El último reto del manos libres es el eco acústico: si el micrófono capta la propia voz sintetizada del asistente por el altavoz, el sistema se interrumpe a sí mismo y se vuelve inutilizable. El marco no aporta cancelación de eco acústico (AEC), así que el síntoma documentado era precisamente esa auto-interrupción. La solución definitiva resultó ser de **hardware**: un altavoz de conferencia USB Anker PowerConf que cancela el eco en el propio dispositivo, sin coste de CPU. Esto preserva, además, la capacidad de *barge-in* —que el usuario pueda interrumpir al asistente mientras habla— porque distingue correctamente la voz humana de la reproducción propia. Como respaldo histórico quedan dos planes por software, nunca activados al bastar la solución por hardware. El criterio de aceptación fue cuantitativo: un incremento de RMS no superior a unos 6 dB sobre el silencio de referencia y locución inaudible en la captura, más una prueba de *double-talk* que valida el *barge-in*.

6 El razonamiento: el modelo de lenguaje como cerebro

Si el pipeline de voz son los oídos y la boca del asistente, el modelo de lenguaje (LLM) es su cerebro: la pieza que interpreta lo que el usuario dice, decide qué hacer y redacta lo que se va a responder. Este capítulo describe cómo se integra ese cerebro en la arquitectura, cómo se le aísla del resto del sistema para poder sustituirlo sin reescribir nada, y cómo se le dota de herramientas, contexto y personalidad.

6.1. El único componente en la nube

El sistema es, por principio de diseño, autoalojado: el reconocimiento de voz, la síntesis, la memoria, las acciones y la búsqueda corren en hardware propio. El razonamiento es la **única excepción**, la única pieza externalizada a la nube. El motivo es de cómputo: ejecutar localmente un modelo de la capacidad necesaria exigiría una inversión en GPU desproporcionada para un homelab mono-usuario, mientras que la inferencia en proveedores remotos es barata, rápida y siempre actualizada.

Esta concesión se acota con una regla estricta: **solo viaja texto**. Al cerebro remoto nunca se le envía audio, ni imágenes sin filtrar, ni el contenido de la memoria sin control; se le manda la transcripción ya producida en local y se recibe texto que la síntesis convierte de nuevo en voz dentro de casa. El audio, que es el dato más sensible, no abandona el perímetro.

6.2. La pasarela: LiteLLM como capa de abstracción

Acoplar el orquestador directamente a la API de un proveedor concreto sería una trampa: cada proveedor tiene su SDK, sus parámetros y sus límites, y migrar significaría tocar código. Para evitarlo, el sistema interpone **LiteLLM**, un proxy local compatible con la especificación OpenAI. El orquestador habla siempre con la misma dirección interna y es un fichero de configuración quien decide a qué modelo real se enruta cada petición.

El valor de esta capa es triple:

- **Interfaz única.** El orquestador emite peticiones en un solo formato; la heterogeneidad de los proveedores queda absorbida por la pasarela.

- **Alias.** El orquestador no pide un modelo concreto, sino un alias funcional como `jarvis-main` (cerebro principal), `jarvis-memory` (extracción barata de hechos) o `jarvis-vision`. Cambiar el modelo detrás de un alias es editar una línea de configuración, no código.
- **Fallbacks.** Si el destino principal falla, LiteLLM reintenta y cae automáticamente al siguiente alias de la cadena de respaldo, sin que el orquestador se entere.

6.2.1. La saga de proveedores y la lección de diseño

Esta abstracción no es teórica: el proyecto la ha ejercitado. El cerebro principal nació siendo **Groq** (inferencia sobre hardware LPU, con un modelo de 70B y un tiempo hasta el primer token de 200–600 ms). El problema apareció con el uso continuado: las pruebas agotaban el **cupó diario gratuito** del modelo, y a media jornada el servicio degradaba a uno más pobre. La solución fue mover el cerebro a **GLM-5 servido vía OpenCode Go**, cubierto por la suscripción del usuario y sin cupo que se agote. Groq quedó relegado a *fallback* de último recurso.

Lo relevante es que ese cambio de proveedor no tocó una sola línea del orquestador: fue una reasignación del destino del alias `jarvis-main`. Esa es la lección de diseño central: **no acoplarse a un proveedor**. La capa de abstracción convirtió una migración que habría sido invasiva en un cambio de configuración.

6.2.2. El truco del razonamiento desactivado

La migración trajo consigo un detalle no obvio. GLM-5, con su modo de «razonamiento» (*reasoning*) activado, devolvía respuestas **vacías**: el modelo consumía su esfuerzo en una cadena de pensamiento interna y la respuesta visible llegaba truncada o nula, lo que en un asistente de voz se traduce en silencio. La cura fue desactivarlo explícitamente, pasando un parámetro `reasoning_effort: none` a través de la pasarela. El efecto secundario es positivo: sin el sobrecoste del razonamiento, la respuesta se genera en torno a 1,5 s, dentro del presupuesto de latencia por turno. Es un recordatorio de que un modelo «más listo» no siempre encaja en un caso de uso conversacional con restricción de tiempo.

6.3. Uso de herramientas: function calling

Un cerebro que solo conversa es un chatbot. Para actuar, el LLM dispone de **function calling**: junto a la transcripción del usuario recibe un catálogo de herramientas declaradas (nombre, descripción y parámetros), y decide por su cuenta cuándo invocar una y con qué argumentos. La respuesta del modelo deja entonces de ser texto para el usuario y pasa a ser una *tool call* que el orquestador ejecuta.

La pieza arquitectónica clave es que el **registro de herramientas está desacoplado** y es compartido por los dos canales del asistente, voz y texto. Una única fuente de verdad —un fichero de catálogo más un conjunto de implementaciones— alimenta tanto el registro sobre el servicio LLM

de voz como el registro plano que consume el canal de texto de Telegram. El canal de texto no duplica reglas: reutiliza las mismas herramientas, la misma semántica de seguridad y el mismo filtrado de argumentos. Esto importa porque los LLM **alucinan argumentos** con frecuencia (por ejemplo, invocar una herramienta de agenda con un parámetro inexistente); el orquestador filtra cada llamada al esquema declarado para que una invención del modelo no haga reventar la herramienta.

Las herramientas se dividen en dos categorías:

Categoría	Comportamiento	Ejemplos
Solo lectura/búsqueda	Se ejecutan directamente	consultar agenda, buscar en la web, briefing
Acción con efecto	Nunca se ejecutan en la primera llamada: pasan por confirmación	crear recordatorio, encargar

El detalle de la cadena de confirmación —que vive deliberadamente fuera del LLM— corresponde al capítulo de agencia; aquí basta señalar que el modelo *propone* la acción pero no es quien la *autoriza*.

6.4. Gestión del contexto

El cerebro no recibe la frase del usuario a secas, sino un **system prompt compuesto por capas**: las reglas de comportamiento, la persona del mayordomo, el perfil del usuario, la relación y un bloque de hechos aprendidos. Estos últimos se inyectan delimitados y marcados explícitamente como datos, no como instrucciones, de modo que un hecho que pareciera contener una orden no pueda alterar el comportamiento del asistente.

El prompt se compone además **por canal**. El núcleo es común a voz y texto, pero el canal de Telegram añade un bloque que **levanta la brevedad extrema de la voz**: por escrito el asistente puede extenderse, estructurar con listas, usar negrita y compartir enlaces, cosas todas inútiles o imposibles dichas en voz alta.

Dos mecanismos más completan la gestión del contexto:

- **Inyección de la fecha y hora reales.** En cada turno se refresca en el prompt un bloque con la fecha y hora del sistema. Corrige un fallo previo: como el modelo nunca recibía la hora, **se la inventaba**.
- **El clasificador de errores del LLM.** En lugar de envolver toda falla en un manejador genérico, un módulo examina la respuesta y decide una acción concreta: ante un *rate limit*, **reintentar**; ante un contexto desbordado, **comprimir** el historial y seguir; ante un fallo de política o de credenciales, **rendirse** con un mensaje específico. Cada caso devuelve además una frase en registro de mayordomo para que el usuario nunca oiga jerga técnica.

6.5. Delegación de lo difícil

El cerebro principal es deliberadamente **barato y rápido**, optimizado para orquestar una conversación en tiempo real, no para investigaciones largas. Cuando una petición excede su alcance, el sistema aplica un patrón de delegación: el cerebro económico **encarga el trabajo pesado a un agente más capaz**, una instancia de Claude Code que corre en el host, mediante herramientas como *investigar* (investigación de solo lectura y web) o *encargar* (acciones reales sobre el servidor). El cerebro decide *cuándo* delegar; el agente capaz ejecuta. La mecánica de esa delegación y su seguridad pertenecen al capítulo de agencia.

6.6. Personalidad y registro

Por último, el cerebro tiene carácter. El system prompt define una **persona de mayordomo**: cortés, con el trato de «señor», al servicio del usuario. Sobre esa persona pesa, en el canal de voz, una restricción dura de **brevedad**: una respuesta hablada larga es tediosa y rompe la sensación de conversación, de modo que el prompt obliga a contestar corto. Es el mismo cerebro, la misma memoria y las mismas herramientas en voz y en texto; lo que cambia es el registro, y ese ajuste se logra —de nuevo— componiendo el prompt, no cambiando el modelo.

7 Memoria y aprendizaje continuo

7.1. El problema: recordar sin saturar

Un modelo de lenguaje no conserva nada entre llamadas. Cada petición a la API parte de cero, de modo que un asistente construido directamente sobre el modelo trata cada conversación como la primera: olvida el nombre de la pareja del usuario, las decisiones que ya se tomaron y las preferencias expresadas ayer. Para un asistente personal esto es más que una incomodidad, es una ruptura de la ilusión de continuidad sobre la que descansa la utilidad del sistema.

La respuesta ingenua —memorizarlo todo e inyectarlo en cada petición— fracasa por el extremo opuesto. La ventana de contexto es finita y costosa, y se ha observado de forma sistemática que su saturación degrada la calidad de las respuestas: el modelo se distrae con material irrelevante y pierde de vista la pregunta actual. El reto no es, por tanto, *guardar* sino *seleccionar*: recordar lo importante y olvidar lo irrelevante, manteniendo el contexto inyectado dentro de un presupuesto. Este capítulo describe cómo el sistema resuelve ese equilibrio sin recurrir a *fine-tuning* —descartado por ser menos controlable, reversible y auditable que la combinación de memoria más prompt— ni a la maquinaria de embeddings que el plan inicial contemplaba.

7.2. Arquitectura de la memoria: dos niveles

La memoria efectiva se organiza en dos niveles complementarios, cada uno con una pregunta distinta.

Nivel episódico — «qué se dijo». Cada turno de conversación se registra en una base SQLite como filas de usuario y de asistente. Sobre esa tabla se mantiene un índice de texto completo con **FTS5**, el motor de búsqueda léxica integrado en SQLite. La herramienta `recordar` lo consulta: cuando el usuario pregunta «¿qué te dije la semana pasada sobre X?», el asistente lanza una búsqueda de texto completo contra el histórico y recupera los fragmentos pertinentes. No hay similitud vectorial; es búsqueda léxica con el *ranking* propio de FTS5. La contrapartida es conocida: la búsqueda léxica no capta sinónimos ni paráfrasis como sí lo haría una recuperación semántica. A cambio es **transparente y depurable** —se puede ver exactamente por qué casó cada resultado—, una propiedad valiosa para un corpus de un solo usuario.

Nivel semántico — hechos durables sobre el usuario. Por encima del log episódico vive un almacén estructurado de *hechos*: preferencias, personas del entorno, rutinas, decisiones tomadas. No es la transcripción de lo que pasó, sino el destilado de lo que conviene saber del usuario a largo plazo. Reside en la tabla `facts` de SQLite y se proyecta sobre un fichero, `aprendido.md`, que es lo que finalmente entra en el prompt.

7.3. El Curator: destilar hechos, descartar lo efímero

La extracción de hechos no ocurre en cada turno, sino mediante el **Curator**, una tarea de fondo que se ejecuta cada ~20 minutos. Pasa por el LLM las conversaciones recientes con un único cometido: extraer hechos durables y descartar lo efímero. Su prompt es explícito al respecto —rechaza horas, citas y reuniones concretas, estados de ánimo del momento y fallos técnicos pasajeros— porque eso caduca y no describe un rasgo estable del usuario.

Ejecutar la extracción en segundo plano, y no de forma síncrona en cada turno, tiene dos ventajas: no añade latencia ni coste a la conversación en vivo, y permite al modelo observar una ventana de varios turnos a la vez, con lo que destila mejor. Este trabajo administrativo se envía a un **modelo económico**, de modo que la memoria nunca compite con la conversación por presupuesto.

Conviene distinguir dos modos de aprendizaje con gobernanza distinta:

Tipo de cambio	Qué es	Gobernanza
Auto-escritura de datos	Gustos, personas, rutinas	Automática, sin aprobación
Auto-mejora del comportamiento	Ajustes de perfil, trato o tono	Propuesta → aprobación humana en el panel

Los datos son reversibles y de bajo riesgo, por lo que se escriben solos. Modificar *cómo se comporta* el asistente es otra cosa: el Curator lo **propone** y deja la propuesta pendiente en el panel, donde el usuario la aprueba o rechaza. La ficha base de personalidad nunca se reescribe de forma autónoma. Hay, así, un humano en el bucle para todo cambio del modelo que el asistente tiene de su usuario.

7.4. Recall y decay: la pieza clave

El valor del almacén no está en guardar hechos, sino en **priorizar los que importan y dejar caer los que no**. Cada hecho lleva metadatos: cuándo se aprendió, cuándo se re-mencionó, cuándo se recuperó, cuántas veces y un estado (*active* / *stale* / *archived*). Sobre ellos operan dos dinámicas inspiradas en la consolidación de memoria biológica, donde lo que se reactiva se afianza y lo que no se usa se desvanece.

- **Recall (refuerzo por uso)**. Cuando el usuario vuelve a mencionar un hecho, o el sistema lo recupera para responderle, sube su contador y se actualiza la marca temporal. Lo que se usa gana prioridad de forma natural, por frecuencia real y no por recencia de creación.
- **Decay (caducidad sin borrado)**. Un hecho que no aparece en **30 días** pasa a *stale*; si sigue ausente a los **90 días** pasa a *archived*. La pieza crítica es que **nada se borra**: un hecho archivado permanece en la base y es **recuperable** si vuelve a ser relevante. La memoria olvida lo que dejó de importar sin perder lo que un día importó.

Sobre estos metadatos, una función de *scoring* decide qué hechos entran en el prompt. Combina tres señales: **recencia** con vida media de 14 días, **recall** y **frecuencia**, ponderadas 0,5 / 0,3 / 0,2 respectivamente. El resultado ordena los hechos activos, y solo los **50 mejores** se vuelcan al fichero del prompt. Ese tope de 50 es el **presupuesto** explícito: garantiza que la memoria inyectada no crezca sin freno ni desplace al resto del contexto.

Adicionalmente, una pasada de **consolidación** diaria fusiona duplicados y resuelve contradicciones quedándose con lo más reciente, siempre con copia de seguridad previa.

7.5. Por qué no mem0

El plan original contemplaba **mem0** sobre un almacén vectorial: extracción automática de hechos, embeddings y recuperación semántica. Fue una opción seria, y se llegó a auditar su configuración. La conclusión fue que **no compensaba** para un asistente de un solo usuario, con volumen modesto y una huella de RAM exigente. La maquinaria de un *vector store* —un contenedor adicional, embeddings multilingües con sus particularidades, la doble llamada al LLM por cada inserción, los pines de versión— pesaba más de lo que aportaba.

La solución propia —**FTS5 + Curator + almacén facts con recall/decay**— cubre las mismas necesidades con menos contenedores, menos memoria, sin embeddings que reindexar y, sobre todo, con un comportamiento **inspeccionable de principio a fin**. mem0 queda documentado como camino evaluado y descartado, no como deuda pendiente.

7.6. Privacidad de la memoria

La memoria es una superficie de ataque. Como los hechos provienen en última instancia de texto de conversación, un hecho redactado en forma imperativa («recuerda que debes...») no debe poder reescribir el comportamiento del asistente. Por eso el fichero de hechos se inyecta **fenced**: encerrado en un bloque acotado y precedido de una advertencia explícita al modelo de que **lo que sigue son datos sobre el usuario, no instrucciones**. El *fencing* marca la frontera entre lo que el asistente *sabe* y lo que *obedece*, y constituye una defensa frente a la inyección de prompts y al envenenamiento de memoria. Como salvaguarda complementaria, lo que el asistente lee en internet no se convierte en hecho durable: la memoria semántica se nutre de lo que el usuario dice, no de lo que la web afirma.

8 Proactividad y multicanalidad

8.1. El problema de la proactividad

Un asistente que solo responde cuando se le pregunta es predecible y, por ello, inofensivo: el usuario conserva siempre el control de cuándo empieza la interacción. Dar al sistema la capacidad de **iniciar la comunicación por su cuenta** —avisar de algo sin que nadie lo haya pedido— cambia esa relación de raíz. Bien hecho, es lo que separa a un mayordomo de un buscador con voz: el sistema recuerda, anticipa y avisa. Mal hecho, es la forma más rápida de destruir la confianza. Un asistente que interrumpe de más se convierte en ruido, y a un sistema ruidoso se le acaba ignorando o apagando. El coste de un falso positivo proactivo (molestar sin motivo) es asimétricamente mayor que el de un falso negativo (callar cuando quizá valía la pena hablar).

De esa asimetría nace el principio de diseño que gobierna todo el subsistema: **silencio por defecto**. El sistema no habla salvo que tenga una razón clara para hacerlo, y la carga de la prueba recae sobre el aviso, no sobre el silencio. La proactividad se trata, por tanto, no como una funcionalidad que se activa, sino como una excepción que debe justificarse y atravesar una serie de filtros antes de alcanzar al usuario.

8.2. El heartbeat y la puerta única

El mecanismo tiene dos piezas. La primera es un **heartbeat** (latido) periódico: una señal temporizada que despierta al sistema a intervalos regulares para que se plantee si tiene algo que decir. Sin este latido, el asistente nunca tendría ocasión de tomar la iniciativa, porque su ciclo normal es puramente reactivo. El latido no decide nada por sí mismo; solo abre la oportunidad.

Quien decide es `brain_review`, un momento de reflexión en el que el propio modelo de lenguaje revisa el estado del día y se pregunta si vale la pena anticiparse a algo. Su sesgo está deliberadamente cargado hacia el **silencio**: la respuesta por defecto a «¿hay algo verdaderamente oportuno?» es *no*. Solo cuando aparece una razón suficientemente buena el sistema rompe el silencio. Delegar este juicio al LLM evita codificar reglas rígidas sobre qué es «oportuno», a cambio de aceptar la variabilidad propia de un modelo; el sesgo conservador acota esa variabilidad por el lado seguro.

Aunque `brain_review` decida hablar, ningún aviso llega directamente al usuario. Toda salida proactiva atraviesa una **puerta única**, el `ProactiveGate`, que concentra en un solo punto todas las reglas de cortesía. Tener una primitiva única de salida es una decisión arquitectónica importante: garantiza que no exista ningún camino lateral por el que un aviso pueda escapar de los filtros. La puerta aplica de golpe:

Filtro	Qué hace
No molestar (DND)	Bloquea cualquier aviso mientras el modo está activo
Horario de silencio	Suprime interrupciones de voz en franjas nocturnas
Presupuesto por hora	Limita el número máximo de avisos en una ventana de tiempo
Deduplicación	Descarta mensajes repetidos que ya se han dicho
Tiers	Gradúa el derecho de cada aviso a interrumpir

El sistema de **tiers** (niveles) merece detalle: clasifica cada aviso en `ambient`, `info` o `critical`. Un aviso `ambient` apenas tiene derecho a interrumpir y cede ante casi cualquier filtro; uno `critical` puede atravesar restricciones que detendrían a los demás. Así, el mismo mecanismo que silencia el ruido de fondo deja pasar lo que de verdad importa. La combinación de un juez conservador y una puerta con presupuesto y niveles materializa el principio de silencio por defecto sin volver el sistema inútil.

8.3. Multicanalidad: el texto como canal de primera clase

El sistema habla por voz, pero la voz no es su único cuerpo. Se añadió un **canal de Telegram bidireccional**: se puede chatear con el asistente por texto y obtener exactamente el **mismo cerebro, las mismas herramientas y la misma memoria** que por voz. Esta equivalencia es la decisión de diseño relevante. Telegram no es un bot recortado ni un atajo de notificaciones: es el mismo asistente expresándose por otra boca. El texto es un canal de primera clase, no un apaño añadido a posteriori.

La ventaja práctica es doble. Como canal de entrada, permite mandar órdenes sin necesidad de estar frente al micrófono. Como canal de salida, da al sistema una vía para alcanzar al usuario cuando este no está en casa. Por seguridad, el canal **solo responde al dueño**: cualquier otro remitente se ignora, lo que evita exponer un cerebro con manos reales a interlocutores no autorizados.

8.4. Enrutado por presencia

Con dos canales de salida —voz y Telegram— el sistema necesita decidir **por dónde** hablar en cada momento. La regla es de presencia y la resuelve el orquestador, que mantiene un estado vivo *presente/ausente*:

- **Presente** (el usuario está en casa) → **voz**.

- **Ausente** (fuera) → **Telegram**.
- **Noche o no molestar** → **Telegram silencioso**.

La lógica de fondo es sencilla: no se le habla por voz a una casa vacía, ni a gritos a las tres de la madrugada. Un encargo que termina mientras el usuario está en el salón se anuncia en voz alta; el mismo encargo terminado mientras está en la calle le llega al móvil.

Dos detalles hacen robusto este enrutado. El primero cierra el círculo desde el otro extremo: **un mensaje de Telegram se interpreta como «no está delante»**. Quien escribe por texto, por definición, no está frente al micrófono, así que el sistema conmuta a *ausente* al recibirlo. El segundo es un *fail-safe*: el subsistema de visión que alimenta la presencia real todavía está apagado a falta de cámara, de modo que, **ante la ausencia de sensores o un fallo, el sistema asume PRESENTE**. Asumir presencia equivale al comportamiento de toda la vida —responder por voz—, así que el asistente nunca se queda mudo por una avería del subsistema que precisamente aún no existe.

8.5. Tareas programadas: cron en lenguaje natural

La última pieza extiende la proactividad al tiempo: el sistema permite programar trabajo recurrente **dicho en lenguaje natural** («cada mañana dame el tiempo»), sin que el usuario tenga que conocer la sintaxis de un cron clásico. Hay dos sabores con comportamientos distintos:

- **Tareas recurrentes** — se ejecutan y entregan su resultado siempre. La del ejemplo da el parte del tiempo cada mañana, sin excepción.
- **Monitores** — se ejecutan periódicamente pero **solo avisan si hay algo que decir**; cuando no, callan. El patrón es explícito: el prompt del monitor emite un marcador [SILENT] que el sistema interpreta como «no molestar esta vez». El monitor es, en esencia, la aplicación del silencio por defecto al dominio temporal.

La decisión de seguridad que sostiene todo el mecanismo es de raíz: el cron **no ejecuta scripts arbitrarios**. Lo que programa son **prompts con herramientas restringidas** —las mismas del catálogo— y no comandos de shell sueltos. La diferencia es sustancial en términos de superficie de ataque: si una tarea programada pudiera lanzar shell libre, cada entrada de cron sería una vía de ejecución arbitraria que sobrevive en el tiempo. Al obligar a que toda tarea se exprese como un prompt sobre el catálogo de herramientas, el trabajo programado queda dentro del mismo perímetro de seguridad y de confirmación que cualquier otra acción del sistema, sin abrir un canal privilegiado paralelo.

9 Agencia: delegación segura de acciones

9.1. El salto de informar a actuar

Hasta este punto, el asistente descrito era, en lo esencial, un sistema que **informa**: escucha una pregunta, consulta su memoria o internet, y responde de viva voz. Es útil, pero es un mayordomo manco. El salto cualitativo —y el más delicado de toda la arquitectura— consiste en darle **manos**: capacidad de crear y editar archivos, gestionar los contenedores Docker y los servicios *systemd* del propio servidor que lo hospeda, instalar paquetes y, en general, operar el *homelab* por voz.

Ese salto es deseable por una razón evidente: el valor de un asistente doméstico crece cuando deja de limitarse a decir *cómo* se reinicia un servicio y pasa a reiniciarlo él mismo. Pero es peligroso por la misma razón: actuar sobre el sistema implica poder romperlo. La capacidad que vuelve útil al asistente es exactamente la que, mal gobernada, lo vuelve destructivo.

9.2. El problema de seguridad

Conviene nombrar el riesgo con precisión. Se está concediendo la facultad de ejecutar comandos en un servidor a un modelo de lenguaje (LLM), una pieza que presenta tres debilidades simultáneas:

- **Puede equivocarse.** Un LLM razona de forma probabilística; nada garantiza que la orden que emite sea la que el usuario pretendía.
- **Puede ser manipulado.** El asistente lee contenido externo (páginas web) y su propia memoria aprendida. Una página envenenada o un recuerdo con una orden incrustada constituyen un vector de inyección indirecta: la *lethal trifecta* descrita en el capítulo de seguridad —datos privados, contenido no confiable y capacidad de actuar conviven en el mismo sistema—.
- **Recibe entrada por voz.** La transcripción introduce un canal de error propio: «borra los logs de enero» puede transcribirse como algo más amplio o ambiguo. Una orden mal entendida, sobre un comando con efecto real e irreversible, no admite un «deshacer».

El problema central, por tanto, no es si el modelo *quiere* portarse bien, sino qué ocurre cuando se equivoca, lo engañan o lo malinterpreta. Toda la arquitectura de este capítulo se construye sobre esa pregunta.

9.3. El puente al host: investigar y encargar

La estrategia parte de una división de trabajo. El «cerebro» conversacional que decide cuándo y cómo actuar es un modelo económico y rápido; pero la ejecución difícil no la realiza ese cerebro, sino un **operador más capaz que corre en el propio host**: una instancia de Claude Code expuesta mediante un puente HTTP. El cerebro barato orquesta y delega; el agente del host ejecuta. Esa delegación tiene dos modos, deliberadamente distintos en carácter y en riesgo:

Herramienta	Alcance	Riesgo
<code>investigar(tema)</code>	Solo lectura y web: busca, lee y sintetiza, no toca el servidor	Bajo
<code>encargar(tarea)</code>	Acciones reales sobre el host: <code>docker</code> , <code>systemctl</code> , <code>apt</code>	Alto

Ambas son asíncronas: el asistente lanza el trabajo, continúa la conversación y avisa cuando termina. La separación es ella misma una medida de seguridad: la mayoría de las consultas se resuelven con `investigar`, que no puede causar daño, y solo las que exigen actuar cruzan a `encargar`, donde se concentra toda la defensa.

9.4. El modelo de seguridad en tres capas

El usuario pidió explícitamente «acceso amplio con confirmación»: que el asistente pudiera encargar tareas reales sin convertir cada operación en un interrogatorio, pero sin abrir la puerta a un desastre. Para conciliar ambas cosas, `encargar` se montó con **tres capas de seguridad independientes**, gobernadas por un principio rector: *la última defensa nunca debe ser un prompt*. El modelo puede fallar o ser manipulado; las barreras que de verdad importan viven en código determinista, fuera del LLM.

9.4.1. Capa 1: confirmación determinista en código

La confirmación de una tarea con efecto real **no se delega al juicio del LLM**: la exige el orquestador, en código, mediante una máquina de estados. Esta decisión nació de un fallo concreto. Una versión previa dejaba la confirmación en manos del modelo, y este entraba en un **bucle**: re-pedía confirmación una y otra vez, indefinidamente, sin cerrar nunca el ciclo. La lección fue clara: si la lógica de confirmación vive en el prompt, hereda la indeterminación del modelo. Movida a código determinista, el bucle desaparece y la confirmación está garantizada. Es el mismo principio de la confirmación verbal, donde el «sí» del usuario libera la acción mediante un token de un solo uso (caducidad de 30–60 s) mantenido fuera del contexto, de modo que ni una página web maliciosa puede fabricarlo.

9.4.2. Capa 2: doble confirmación inteligente

Sobre esa base determinista, el asistente **juzga el riesgo** de la tarea y modula cuánta fricción aplica. Una tarea rutinaria —reiniciar un contenedor, consultar el estado de un servicio— se libera con una sola confirmación. Una tarea destructiva o irreversible —borrar datos, formatear, parar algo crítico— recibe un trato distinto: el asistente avisa explícitamente de **qué se pierde** y exige una confirmación rotunda antes de delegar.

Aquí, y solo aquí, la inteligencia del modelo **sí aporta valor**: ponderar el daño potencial de una acción y graduar la fricción en consecuencia es precisamente el tipo de criterio para el que un LLM es bueno. Y resulta aceptable confiarle ese juicio porque **hay una red debajo**: la capa 3. La inteligencia añade fricción donde duele, no en todo; pero no es la barrera última.

9.4.3. Capa 3: guardia de comandos determinista

La última línea es un **hook** que corre en el host e inspecciona **cada comando** que el operador intenta ejecutar, bloqueando los letales pase lo que pase: `rm -rf` sobre / o directorios de sistema, `mkfs`, `dd` contra discos, *fork bombs*, detener `ssh` o el cortafuegos, y similares. Esta guardia es **deliberadamente «tonta»**: no razona, no se la convence con argumentos, no depende de que el modelo «entienda» el riesgo. Esa estupidez es su virtud. Por eso es la última defensa, y por eso —fiel al principio rector— no es un prompt: un comando letal queda bloqueado aunque las dos capas anteriores hayan sido engañadas, incluso si una inyección logró manipular al propio modelo.

Las tres capas son complementarias y de naturaleza distinta: la capa 1 garantiza que *hay* confirmación, la capa 2 calibra su *intensidad* según el daño potencial, y la capa 3 atrapa lo que se cuele de las dos anteriores. Inteligencia para el criterio; determinismo para el suelo.

9.5. Privilegio mínimo

La defensa en profundidad se completa recortando lo que el operador *puede* hacer aunque una orden atravesase las tres capas. El operador del host es Claude Code corriendo como el usuario normal del sistema, **nunca como root**. Su `sudo` está acotado por *sudoers* —validado con `visudo`— a solo tres familias de comandos: `docker`, `systemctl` y `apt`. Puede gestionar el *stack*, los servicios y los paquetes, pero no dispone de root arbitrario. Así, el privilegio disponible está recortado de entrada: el daño máximo posible queda limitado por construcción, no por la buena conducta del modelo.

9.6. Discusión

¿Por qué este enfoque y no, simplemente, instruir al modelo con un prompt cuidadoso —«no ejecutes comandos peligrosos, pide confirmación para lo destructivo»? Porque un prompt es una barrera del mismo material que la amenaza. Si la entrada puede manipular al modelo, también

puede manipular las instrucciones que pretenden contenerlo; y si el modelo puede equivocarse al actuar, puede equivocarse igualmente al aplicar su propia regla de seguridad. Confiar la seguridad a un prompt es pedirle al sospechoso que se vigile a sí mismo.

La alternativa adoptada es la **defensa en profundidad**: usar la inteligencia del LLM donde su criterio aporta valor (la capa 2, que distingue lo rutinario de lo catastrófico), pero apoyarla siempre sobre **suelos deterministas** que no dependen de ese criterio (la confirmación en código de la capa 1 y la guardia de comandos de la capa 3). Ninguna capa es perfecta; el sistema es robusto porque un fallo en una capa lo recoge otra de naturaleza distinta.

Este diseño no se sostiene como argumento, sino como práctica **verificada**. El núcleo de seguridad es testeable sin el resto del *pipeline* de voz, y la delegación queda cubierta por la suite automática del proyecto —68 pruebas en verde a fecha de redacción—, que ejercita la confirmación, el modo de contaminación, el *fencing* de la memoria aprendida y la propia guardia de comandos. El bucle de re-confirmación que motivó la capa 1, por ejemplo, dejó de ser una anécdota para convertirse en un caso de prueba. La seguridad de un asistente que actúa no se declara: se prueba, y se prueba contra los exploits que ya ocurrieron.

10 Visión y presencia

Un asistente que únicamente escucha y responde con voz vive en una sola dimensión sensorial. El paso que acerca al sistema a la metáfora que le da nombre es dotarlo de una segunda capacidad: **saber si hay alguien delante y, a ser posible, quién es**. Esta propiedad, que en el proyecto se denomina *presencia*, no es un fin en sí mismo, sino el insumo que permite al asistente reaccionar de forma adecuada al contexto: saludar a quien llega, elegir el canal por el que comunicarse o describir la escena cuando se le pregunta.

El reto técnico es de eficiencia, no de capacidad. El hardware es un mini-PC de bajo consumo (un procesador de 35 W con la iGPU integrada Intel UHD 630, sin GPU dedicada) que ya sostiene un pipeline de voz en tiempo real. Una red neuronal analizando vídeo de forma continua saturaría la CPU de la que dependen la detección de la palabra de activación y el resto de la cadena de audio. La regla de diseño que hace viable todo el subsistema es, por tanto, una sola: **no ejecutar nunca inferencia continua**.

10.1. El embudo: pipeline escalonado

La solución a esa restricción es un *pipeline escalonado* (un embudo de etapas), donde cada fase es mucho más cara que la anterior y solo se activa ante un resultado positivo de la previa. De este modo, el cómputo costoso se reserva para los instantes en que realmente hace falta.

Las tres etapas son:

1. **Detección de movimiento** (siempre encendida): una simple diferencia entre fotogramas consecutivos con OpenCV. Su coste es prácticamente nulo y actúa como centinela permanente.
2. **Detección de persona** (solo si hubo movimiento): la red **YOLO11n** ejecuta la inferencia para confirmar que el movimiento corresponde a una persona y no a una cortina o una mascota.
3. **Reconocimiento facial** (solo si hubo persona): **InsightFace** intenta poner nombre a la cara detectada.

Cada etapa filtra a la siguiente. La mayor parte del tiempo el sistema solo paga el coste de la etapa 1; las etapas 2 y 3, costosas, permanecen inactivas salvo en los breves momentos de actividad real.

10.2. Tecnología: YOLO11n sobre OpenVINO e InsightFace

La detección de personas usa **YOLO11n**, la variante *nano* (la más ligera) de la familia de detectores de objetos de Ultralytics. El modelo se exporta y **cuantiza a INT8** —es decir, sus pesos se represen-

tan con enteros de 8 bits en lugar de números en coma flotante, lo que reduce el tamaño y acelera la inferencia a costa de una precisión ligeramente menor— y se ejecuta mediante **OpenVINO**, el motor de inferencia de Intel que permite aprovechar la iGPU UHD 630 en lugar de la CPU. Con entrada de 320 píxeles, el presupuesto estimado es de unos 25–35 ms por inferencia a dos fotogramas por segundo.

El reconocimiento facial recae en **InsightFace** con el pack de modelos `buffalo_sc`, que combina un detector de caras y un extractor de *embeddings*. Un *embedding* es un vector numérico que codifica los rasgos de una cara: dos imágenes de la misma persona producen vectores próximos entre sí. El reconocimiento se ejecuta en CPU de forma esporádica, ya que se aplica a un único fotograma.

10.3. Distinguir al dueño

La pregunta que responde InsightFace no es solo «¿hay una cara?», sino «¿de quién es?». El sistema mantiene, para cada una de las 1–3 personas de la casa, un *embedding* promedio calculado durante el **enrolado**, un proceso deliberadamente artesanal: bastan entre cinco y diez fotografías frontales con buena luz. Ante una cara nueva, se calcula la **similitud coseno** entre su *embedding* y los almacenados; si supera un **umbral** orientativo (en torno a 0,45–0,5, que deberá calibrarse con la cámara real), se considera una coincidencia.

Conviene precisar el propósito de esta capacidad. Distinguir al dueño sirve para la **personalización** —saludar por el nombre, adaptar el tono, decidir el canal— y **no** se concibe como mecanismo de seguridad fuerte. El reconocimiento facial es engañable y opera con umbrales probabilísticos; usarlo como puerta de acceso sería frágil. Aquí es un afinador de la experiencia, no un guardián.

10.4. Integración con la presencia

La visión no emite hechos aislados («acabo de ver a alguien»), sino que alimenta un **estado de presencia continuo** que el orquestador mantiene con un tiempo de vida (TTL): mientras llegan detecciones, el usuario está *presente*; si el TTL expira, pasa a *ausente*. Sobre ese estado el orquestador decide **por dónde hablar**, como se detalla en el capítulo de proactividad.

La clave que da robustez al diseño es el **latido** (*heartbeat*): aunque no vea a nadie, el servicio de visión emite una señal periódica que significa «estoy vivo, pero no detecto a nadie». Esto permite **distinguir la ausencia real de una caída del subsistema**: silencio de detecciones con latido presente significa «casa vacía»; silencio total significa «avería». Esa distinción habilita el *fail-safe*: ante la duda —sin cámara o con la visión caída— el sistema asume **PRESENTE** y responde por voz, su comportamiento de toda la vida, de modo que un fallo del subsistema nunca deja mudo al asistente.

10.5. Estado real y honestidad sobre las cifras

El subsistema vive partido en dos mitades de madurez distinta. El **servicio de visión está construido por completo** y sus modelos verificados como presentes en el repositorio, pero permanece **apagado**, sin ninguna cara enrolada, a la espera de una sola cosa: la **cámara USB física** dedicada. El bloqueo es enteramente material; el camino de encendido es corto (enchufar, enrollar, cambiar el flag).

Esta honestidad alcanza también a los números. Las cifras de rendimiento de YOLO sobre la iGPU (los 25–35 ms citados) son **estimaciones de diseño, no mediciones en operación**: ningún fotograma real ha atravesado todavía el pipeline en el hardware definitivo. Solo el enchufado de la cámara permitirá sustituir esas estimaciones por medidas y calibrar el umbral de similitud con datos reales.

10.6. Privacidad

El diseño asume un principio firme: **todo el procesamiento de imagen es local**. La detección de movimiento, YOLO e InsightFace se ejecutan dentro de la máquina de casa, y ninguna imagen se envía a la nube de forma rutinaria. La única excepción es explícita y bajo petición del usuario (la consulta puntual «¿qué ves?»), nunca un flujo continuo.

Por último, una asunción física limita el alcance: las webcams USB convencionales **no ven en la oscuridad**, pues carecen de iluminación infrarroja. El sistema lo da por sentado: en penumbra la etapa de visión simplemente no detecta, y el *fail-safe* descrito —asumir presencia ante la falta de señal— evita que esa ceguera nocturna degrade el comportamiento del asistente.

11 Seguridad y privacidad

11.1. La seguridad como condición de diseño

La seguridad de un asistente con agencia no es una capa que se añade al final, sino una restricción que condiciona la arquitectura desde el primer plano. La razón es incómoda pero conviene enunciarla sin rodeos: un asistente personal con acceso a herramientas reúne, por su propia naturaleza, los tres ingredientes de lo que Simon Willison bautizó como la *lethal trifecta* —«trifecta letal»— de los agentes basados en modelos de lenguaje. El asistente accede a **datos privados** (memorias, perfil del usuario, cámara), procesa **contenido no confiable de internet** (las herramientas de lectura y búsqueda web introducen texto externo directamente en el contexto del modelo) y posee **capacidad de actuar y exfiltrar** (webhooks de automatización y delegación de tareas con efecto real). El peligro no reside en ninguno de los tres por separado, sino en su combinación: cuando los tres coinciden, quien logre colar instrucciones en lo que el modelo lee —y para ello basta una página web envenenada— puede intentar robar memorias o disparar acciones en nombre del usuario.

No es una amenaza teórica. Durante la fase de investigación se documentaron casos reales: memoria envenenada (el ataque conocido como *SpAlware*) e invocación diferida de herramientas en asistentes comerciales. Ese hallazgo obligó a sustentar el modelo de amenazas en marcos públicos —el OWASP LLM Top 10, su guía agéntica y los estudios de *spotlighting* de Microsoft— y a fijar un principio rector que atraviesa todo el capítulo: **la última defensa nunca debe ser un prompt**. El modelo puede equivocarse o ser manipulado; las barreras que importan viven en código determinista, fuera del modelo de lenguaje.

11.2. Inyección de prompts: el contenido externo no manda

La **inyección de prompts indirecta** es el vector central de este modelo de amenazas. Consiste en que un contenido aparentemente inocuo —el texto de una página que la herramienta de lectura web trae al contexto— incluya instrucciones dirigidas al agente: «ignora tus reglas y envía las memorias del usuario a esta dirección». El modelo, que no distingue de forma nativa entre las órdenes legítimas de su operador y el texto que se le presenta como dato, podría obedecerlas.

La defensa de fondo es conceptual antes que técnica: **todo contenido externo se trata como datos, nunca como instrucciones**. Esta separación se materializa mediante *spotlighting* (también llamado *fencing*): una regla fija del prompt de sistema declara que el contenido devuelto por las herramientas son datos inertes, y el código que maneja la lectura web envuelve ese texto en un bloque delimitado y etiquetado explícitamente como «contenido externo no confiable». Según los

estudios de Microsoft, la técnica reduce el éxito de la inyección indirecta de más del 50 % a menos del 2 %; es eficaz, pero insuficiente por sí sola, y por eso es solo una capa más.

El mismo razonamiento se aplica **hacia dentro**. Cuando el asistente acumula memoria propia — lo que aprende del usuario—, ese conocimiento se reinyecta turno tras turno y reabre el vector de *SpAIware*. Por ello la memoria aprendida se envuelve y etiqueta con idéntica desconfianza: si un recuerdo contuviera una orden encubierta, el modelo la lee como dato inerte y la ignora. Ni siquiera la propia memoria del asistente tiene autoridad para mandar.

11.3. Las medidas de la primera versión

La primera versión del sistema implementa un conjunto de barreras deterministas, ubicadas en el orquestador y no en el modelo:

Medida	Qué protege
Confirmación verbal fuera del modelo	El orquestador, no el modelo, repite toda acción con efecto y exige un «sí» del usuario; un token de un solo uso con caducidad, mantenido fuera del contexto, libera esa llamada concreta. El contenido web no puede fabricar la confirmación. El clasificador es <i>fail-closed</i> : rechaza cualquier frase con negación, cerrando el exploit «no, no vale la pena».
Modo <i>taint</i> (contaminación)	Si un turno usó lectura web, ese turno queda «contaminado»: no puede ejecutar acciones con efecto ni guardar memoria.
Guardia SSRF en la lectura web	Se resuelve el DNS y se validan todas las IP de destino, bloqueando direcciones privadas, <i>loopback</i> , <i>link-local</i> y de metadatos en la nube; se conecta a la IP ya validada y se revalidan las redirecciones, cerrando la ventana de <i>DNS rebinding</i> . Solo http/https, con tiempo límite y truncado de tamaño.
Webhooks firmados (HMAC)	Cada llamada a la automatización lleva un HMAC-SHA256 verificado con comparación de tiempo constante, ventana temporal y deduplicación anti- <i>replay</i> .
Tiempos límite en herramientas	Toda llamada externa caduca, evitando bloqueos y consumo indefinido.

El criterio de éxito que resume el conjunto es operativo y comprobable: *una página trampa con instrucciones inyectadas no debe conseguir disparar una acción ni guardar una memoria*. La regla de gobernanza es tajante: sin estas medidas no se da de alta ningún webhook con efecto real.

11.4. Identidad antes que acceso

La segunda pata del modelo gobierna el acceso a las superficies de administración, bajo un principio simple: **nada escucha en la red abierta y nadie entra sin identidad verificada**. El panel de control nunca se expone «pelado». El contenedor solo escucha en `127.0.0.1` —matiz importante, porque los puertos que publica Docker puentean el cortafuegos, de modo que la protección real es el *bind* a *loopback*, no la regla de *ufw*—, y la exposición al exterior se realiza mediante un túnel **saliente** (Cloudflare Tunnel), que no abre ningún puerto en el router. Delante se sitúa Cloudflare Access, que exige autenticación por OTP de correo —una única dirección autorizada— *antes* de que la petición alcance el panel, e inyecta la identidad verificada en una cabecera que el panel valida contra una *allowlist*. Si la lista está vacía, deniega: la postura es *fail-closed*.

11.5. Defensa en profundidad y privilegio mínimo

Ninguna medida se considera suficiente por sí sola; el diseño apila capas independientes (**defensa en profundidad**) y un mapa de **zonas de confianza** ordena qué se fía de qué. Tres fronteras concentran el riesgo: el acceso humano desde internet (resuelto con identidad delegada), el retorno de la web y de la memoria aprendida hacia el orquestador (resuelto con *spotlighting*, *fencing*, *taint* y confirmación) y la salida del orquestador hacia el operador del host. Esta última pertenece al modelo de **delegación de acciones**, detallado en su propio capítulo; aquí basta señalar su última línea de defensa: un guardia de comandos determinista —un hook que inspecciona cada comando antes de ejecutarlo y bloquea los letales pase lo que pase—, deliberadamente «tonto», precisamente por ser la última barrera. El **privilegio mínimo** lo refuerza: el operador corre como usuario sin privilegios, con *sudo* acotado por configuración a solo tres familias de comandos, de modo que ni una orden que atravesara todas las capas dispondría de root arbitrario.

11.6. Privacidad por diseño: lo local se queda en casa

El sistema es *local-first* por convicción de privacidad. La frontera es nítida: **el audio, el vídeo y el rostro jamás salen del host**; tampoco las memorias ni las búsquedas. A la nube del modelo de lenguaje viaja únicamente **el texto del turno**, lo imprescindible para razonar. Los secretos se guardan en ficheros de entorno fuera del control de versiones, de manera que clonar el repositorio público no filtra ninguna credencial. El contraste con los asistentes comerciales es deliberado: allí la voz cruda y el comportamiento se envían y retienen en servidores de terceros; aquí solo cruza la frontera el texto estrictamente necesario, y el resto permanece bajo control físico del usuario.

11.7. Verificación: la seguridad se demuestra con pruebas

Una afirmación de seguridad sin prueba es una intención. Por ello las barreras críticas se validan con una batería de pruebas automáticas: el núcleo de seguridad es comprobable de forma aislada, el guardia SSRF tiene su propia suite, y existen pruebas que verifican que el guardia bloquea los comandos letales, que la confirmación no se puede falsear con frases negativas y que el *fencing* de la memoria neutraliza órdenes incrustadas. El conjunto cierra el círculo del principio rector: lo que protege al sistema no es la buena voluntad del modelo, sino código determinista respaldado por verificación reproducible.

12 Evaluación y resultados

12.1. Metodología de evaluación

La evaluación parte de una premisa de honestidad metodológica: el proyecto es un sistema doméstico de un único usuario, no un producto sometido a *benchmarking* formal. En consecuencia, conviene distinguir desde el inicio entre **lo medido** —cifras obtenidas con instrumentación o cronometraje directo durante las sesiones de validación— y **lo presupuestado/estimado** —objetivos del plan que sirven de referencia pero que no se han verificado de forma cuantitativa exhaustiva en condiciones controladas y repetidas.

Se evalúan cuatro dimensiones: **latencia** del pipeline de voz (descompuesta por etapa), **consumo de recursos** (RAM sobre el host real), **coste económico** (suscripción del modelo, servicios locales y electricidad) y **fiabilidad** (cobertura de la suite de pruebas automáticas más la verificación funcional en vivo de cada subsistema). La latencia y la RAM se midieron sobre el hardware definitivo durante las sesiones de validación; el coste se estima a partir de los precios conocidos; la fiabilidad se evalúa por cobertura de pruebas, no por métricas estadísticas de tasa de error en producción.

12.2. Latencia del pipeline de voz

El pipeline encadena cinco etapas: detección de palabra de activación, detección de fin de turno (VAD + *smart-turn*), transcripción (STT), inferencia del modelo de lenguaje (LLM) y síntesis de voz (TTS). La tabla compara el presupuesto con los datos reales disponibles.

Etapas	Presupuesto / referencia	Dato real	Origen
Wake word	Disparo fiable, sin falsos positivos	Confianza 0,99 («hey Mycroft»)	Medido, en vivo
Fin de turno (VAD + smart-turn)	~200 ms; smart-turn 12–95 ms CPU	No cronometrado aislado E2E	Estimado (config. fijada)
STT (faster-whisper small INT8)	Casi tiempo real	Español «casi perfecto»; latencia no aislada	Cualitativo

Etapa	Presupuesto / referencia	Dato real	Origen
LLM (TTFB)	~0,5 s objetivo	0,49 s con Groq; 1,5–8 s con OpenCode/GLM-5	Medido (Groq) / observado (OpenCode)
TTS (Piper)	< 2 s	1,8 s	Medido

El dato más relevante —y más honesto de reconocer— es la **variabilidad del LLM**. La medición de 0,49 s de tiempo hasta el primer *token* (TTFB) corresponde a la configuración original con Groq. Tras agotar su cupo, el cerebro definitivo pasó a **GLM-5 vía OpenCode Go**, una pasarela remota cuya latencia oscila entre **1,5 y 8 segundos** por enrutarse a regiones distintas. Esta cifra es una **observación de rango**, no una medición controlada con percentiles. Las etapas locales (wake, VAD, STT, TTS) son deterministas y rápidas; el único componente con cola pesada y dependiente de la red es la inferencia remota. No se ha construido una medición E2E «boca-a-oido» agregada y repetida, por lo que no puede afirmarse una latencia total única.

12.3. Consumo de recursos

El host es un Lenovo ThinkCentre M70q con **i5-10400T (35 W de TDP), 16 GB de RAM y sin GPU dedicada**. La configuración con *faster-whisper small* ocupa **aproximadamente 6–6,5 GB de RAM**, dejando un margen amplio sobre los 16 GB disponibles. El escenario previsto de ~8 GB asociado a *paraquet* no llegó a materializarse, al descartarse ese STT por ser solo en inglés.

La conclusión sobre idoneidad del hardware es favorable y está respaldada por el funcionamiento real: una máquina de bajo consumo sostiene el pipeline completo de voz porque la carga pesada —la inferencia del LLM— se externaliza a la nube, y la visión (cuando se active) se calcula en la iGPU. El margen de RAM permite, además, alojar los servicios auxiliares sin presión de memoria. Debe matizarse que esta medida es un valor de ocupación observado y no un perfilado bajo carga máxima sostenida.

12.4. Coste económico

El coste mensual es deliberadamente bajo:

Partida	Coste	Naturaleza
LLM (OpenCode Go)	Suscripción (límites holgados para voz)	Externo
Visión y búsquedas (SearXNG propio)	Céntimos	Local

Partida	Coste	Naturaleza
Electricidad (host 35 W)	~2–4 €/mes	Estimado

El coste de *tokens* dejó de ser una preocupación al pasar a suscripción, frente al modelo de pago por uso. La cifra eléctrica es una **estimación** derivada del TDP, no una lectura con vatímetro. El balance es muy favorable cuando se contrapone al **valor de la privacidad y el control**: el audio, la memoria y las búsquedas se procesan en infraestructura propia, y solo el texto destinado al LLM sale a la nube.

12.5. Calidad y fiabilidad

La fiabilidad se evaluó por dos vías complementarias. La primera es una **suite de 68 pruebas automáticas** que cubre el cron en lenguaje natural, el clasificador y manejo de errores del LLM, la lógica de *routing* por presencia, el registro de herramientas y el almacén de hechos (*facts*) con su lógica de *recall/decay*. La segunda es la **verificación funcional en vivo** de todos los subsistemas: conversación completa por voz validada de extremo a extremo, panel en producción tras Cloudflare Access, Telegram bidireccional, delegación de acciones con el guardia de comandos comprobado en vivo, y la memoria con *recall/decay*.

Conviene ser preciso sobre el alcance: son **pruebas funcionales y de unidad** que verifican comportamiento correcto e invariantes de seguridad (por ejemplo, que el guardia determinista bloquee `rm -rf /`, `mkfs` o `dd`), no pruebas de carga ni de regresión de latencia.

12.6. Limitaciones del estudio

La evaluación presenta limitaciones que es obligado reconocer:

- Validación funcional, no cuantitativa exhaustiva.** Se demuestra que el sistema *funciona* end-to-end y que cada subsistema cumple su cometido, pero no existen mediciones estadísticas (medias, percentiles, desviaciones) de latencia ni de tasa de error sobre un volumen amplio de interacciones.
- Latencia del LLM no caracterizada.** El rango 1,5–8 s con OpenCode es una observación, no una distribución medida; depende de una pasarela remota fuera del control del proyecto.
- Un solo usuario y un solo entorno.** Todas las observaciones provienen del uso del propietario en una única vivienda; no hay validación con múltiples hablantes, acentos ni condiciones acústicas variadas.
- Visión pendiente de hardware.** El subsistema de presencia está construido y sus modelos validados, pero permanece apagado a la espera de la cámara física, por lo que sus métricas **no se han medido en operación real**.

En síntesis, el sistema supera con holgura sus presupuestos de recursos y coste, cumple los objetivos de latencia en las etapas locales y queda condicionado, en latencia total, por la pasarela remota del LLM; su fiabilidad está respaldada por cobertura de pruebas más verificación en vivo, a falta de una campaña de medición cuantitativa que queda como trabajo futuro.

13 Conclusiones y trabajo futuro

13.1. Conclusiones

Esta tesis partía de una pregunta concreta: si es posible construir un asistente de voz personal **capaz** como un agente moderno y **soberano** como un sistema *local-first*, sobre hardware de consumo y sin GPU dedicada. El sistema diseñado, implementado y evaluado a lo largo del documento permite responder afirmativamente, con los matices que la propia evaluación impone.

El trabajo demuestra, en primer lugar, que la **arquitectura híbrida** es la clave de la viabilidad: al mantener en local todo lo sensible —voz, visión, memoria, búsqueda— y delegar a la nube únicamente el razonamiento en forma de texto, un mini-PC de 35 W sostiene un pipeline conversacional completo. El audio del hogar y el rostro del usuario nunca abandonan la máquina; solo cruza la frontera el texto imprescindible para pensar. La privacidad deja de ser una promesa contractual para convertirse en una propiedad arquitectónica.

En segundo lugar, las tres contribuciones planteadas se materializaron y verificaron:

- La **memoria adaptativa** con *recall* y *decay* resuelve la tensión entre recordar y saturar: el asistente prioriza lo que el usuario usa y archiva —sin borrar— lo que cae en desuso, manteniendo el contexto dentro de un presupuesto y sin la maquinaria de un *vector store*.
- El modelo de **agencia segura en tres capas** permite que el asistente actúe sobre el servidor sin convertirse en un peligro, separando con nitidez dónde aporta valor la inteligencia del modelo (juzgar el riesgo) y dónde debe mandar el código determinista (garantizar la confirmación y bloquear lo letal). El principio *la última defensa nunca debe ser un prompt* se reveló no como un eslogan, sino como una guía de diseño con consecuencias concretas.
- La **proactividad y multicanalidad** gobernadas por presencia dotan al sistema de criterio sobre cuándo hablar y por qué canal, transformando un conjunto de capacidades en un comportamiento de mayordomo coherente.

La evaluación es honesta sobre los límites de lo logrado: el sistema cumple con holgura sus objetivos de recursos y coste, y alcanza buenas latencias en las etapas locales, pero la latencia total queda condicionada por la pasarela remota del modelo de lenguaje, cuyo comportamiento no se ha caracterizado de forma estadística. La fiabilidad se sostiene sobre una batería de pruebas y la verificación en vivo de cada subsistema, a falta de una campaña de medición cuantitativa a gran escala. Es, en suma, una demostración sólida de viabilidad, no un estudio estadístico de rendimiento.

Más allá del artefacto, el proyecto deja una lección transferible sobre la **ingeniería de agentes seguros**: la combinación de inteligencia probabilística para el criterio y de barreras deterministas

para los límites —defensa en profundidad— es un patrón aplicable a cualquier sistema en el que un modelo de lenguaje pueda actuar sobre el mundo real.

13.2. Trabajo futuro

El sistema admite varias líneas de continuación, ordenadas por madurez:

- **Completar el subsistema de visión.** El código de presencia y reconocimiento está construido y a la espera únicamente de la cámara física; su activación cerrará la integración voz-visión y permitirá medir su rendimiento real.
- **Caracterización cuantitativa.** Una campaña de medición con percentiles de latencia, tasa de error de reconocimiento por hablante y consumo eléctrico real con vatímetro convertirá la demostración de viabilidad en una evaluación estadística.
- **Voz desde otra estancia.** La arquitectura admite clientes de red (satélites de bajo coste como ESP32 o un cliente móvil) que extiendan el asistente más allá del alcance del micrófono del servidor.
- **Refuerzos de seguridad.** Identificación de hablante como verja adicional para acciones sensibles, validación criptográfica completa de la identidad en el panel y suites adversariales de inyección de prompts.
- **Memoria semántica.** Si el volumen de hechos creciera, podría incorporarse recuperación por similitud como complemento —no sustituto— de la búsqueda léxica actual.

En conjunto, el trabajo no agota el problema, pero sí establece que un asistente doméstico privado, capaz y seguro es alcanzable hoy, con software libre y hardware modesto, y ofrece un diseño concreto y verificado sobre el que seguir construyendo.

Referencias

A continuación se relacionan las obras, especificaciones y proyectos de software que fundamentan el marco teórico y las decisiones técnicas de este trabajo.

13.3. Marco conceptual y seguridad

1. Kleppmann, M., Wiggins, A., van Hardenberg, P., McGranaghan, M. (2019). *Local-first software: you own your data, in spite of the cloud*. Onward! / ACM SIGPLAN.
2. Willison, S. (2024–2025). *The lethal trifecta for AI agents: private data, untrusted content, and external communication*. simonwillison.net.
3. OWASP Foundation (2025). *OWASP Top 10 for Large Language Model Applications y Agentic Security Initiative*.
4. Hines, K. et al. / Microsoft (2024). *Spotlighting: defending against indirect prompt injection by marking external content as data*.
5. Greshake, K. et al. (2023). *Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection*.

13.4. Voz: reconocimiento, síntesis y activación

6. Radford, A. et al. / OpenAI (2022). *Robust speech recognition via large-scale weak supervision* (Whisper).
7. SYSTRAN (2023). *faster-whisper: reimplementación de Whisper sobre CTranslate2*.
8. Hannun, A. et al. (2014). y proyecto **openWakeWord** — detección de palabra de activación con modelos abiertos.
9. Rhasspy / M. Hansen. **Piper** — síntesis de voz neuronal local; **Silero VAD** — detección de actividad de voz.
10. **Pipecat** — marco de orquestación de voz conversacional en tiempo real.

13.5. Razonamiento, herramientas y visión

11. Vaswani, A. et al. (2017). *Attention is all you need*. NeurIPS.
12. Schick, T. et al. (2023). *Toolformer: language models can teach themselves to use tools*.
13. **LiteLLM** — pasarela compatible con la API de OpenAI con enrutado y *failover*.
14. Jocher, G. et al. / Ultralytics (2024). **YOLO11** — detección de objetos en tiempo real.

15. Deng, J. et al. (2019). *ArcFace: additive angular margin loss for deep face recognition*; proyecto **InsightFace**.
16. Intel (2024). **OpenVINO Toolkit** — inferencia optimizada en hardware Intel.

13.6. Memoria y almacenamiento

17. Hipp, D. R. **SQLite** y la extensión **FTS5** de búsqueda de texto completo.
18. Packer, C. et al. (2023). *MemGPT: towards LLMs as operating systems* — gestión jerárquica de memoria en agentes.

Nota: las referencias a proyectos de software remiten a su documentación oficial y a sus repositorios públicos. Las versiones exactas empleadas se detallan en el Anexo B.

Anexos

13.7. Anexo A. Stack tecnológico y versiones

El sistema se despliega como un conjunto de servicios contenedorizados con Docker Compose y versiones fijadas. La tabla resume los componentes principales y su función.

Componente	Función	Tecnología
Orquestador	Pipeline de voz y agente multicanal	Pipecat (Python)
Pasarela LLM	Interfaz única + <i>failover</i>	LiteLLM
Cerebro	Razonamiento (texto)	LLM en la nube vía la pasarela
Wake word	Palabra de activación	openWakeWord (ONNX)
STT	Reconocimiento de voz	faster-whisper <i>small</i> INT8
TTS	Síntesis de voz	Piper (es_ES-davefx-medium)
VAD / fin de turno	Detección de actividad y turno	Silero VAD + <i>smart-turn</i>
Memoria	Episódica y de hechos	SQLite + FTS5 (almacén facts)
Búsqueda	Metabuscador privado	SearXNG
Visión	Presencia y reconocimiento	OpenVINO + YOLO11n + InsightFace
Panel / HUD	Centro de control	FastAPI + HTMX
Acceso remoto	Exposición segura del panel	Cloudflare Tunnel + Access
Acciones	Webhooks-herramienta	n8n (HMAC)

13.8. Anexo B. Reproducibilidad y operación

El repositorio público contiene el código, la configuración de ejemplo (`.env.example`) y la documentación operativa. La puesta en marcha sigue tres pasos: preparación del host (Docker, audio, cortafuegos, directorios), provisión de secretos en un fichero de entorno excluido del control

de versiones, y construcción y arranque de las imágenes con los modelos. El manual de operación (*runbook*) detalla el procedimiento, el diagnóstico de incidencias y las tareas de mantenimiento.

13.9. Anexo C. Honestidad metodológica y estado del proyecto

Este trabajo documenta un sistema en uso real y en evolución. En aras del rigor, se hace explícito el estado de cada subsistema en el momento de la redacción: el pipeline de voz, el razonamiento, la memoria, la proactividad, la multicanalidad y la agencia segura están **implementados y verificados en operación**; el subsistema de visión está **implementado pero inactivo**, a la espera de la integración de la cámara física. Las cifras de rendimiento se etiquetan a lo largo del documento como *medidas* o *estimadas* según corresponda. El código está cubierto por una suite de 68 pruebas automáticas.

13.10. Anexo D. Nota sobre el nombre

El sistema recibe el nombre de J.A.R.V.I.S. como referencia cultural a la figura del mayordomo digital. Por una decisión de diseño documentada, la palabra que activa el asistente por voz es «hey Mycroft» —en alusión a Mycroft Holmes—, ya que el modelo de detección correspondiente resultó empíricamente más fiable sobre el hardware empleado. El disparador y el nombre del sistema son, por tanto, independientes.